

Modern C++ 工程学习手册

G6 内存与性能 · G7 并发与内存模型

CPP-PILOT-G6-G7-COMPACT

vpilot.1

ACCEPTED CONTENT / PILOT PDF

权威内容来源为 Markdown。本件仅供阅读与评审，未获正式发布批准。

阅读与版本说明

PREVIEW / DRAFT · 非权威阅读视图

本件从已接受的 Markdown 冻结提交取源。正文中的“待审核”保留提交时的历史文字；内容接受不等于 PDF 或正式发布批准。标题、章节编号、代码与 Gate 不因出版改写。封面、目录和代码续页标签是生成的阅读辅助。

制品标识 CPP-PILOT-G6-G7-COMPACT

版本 pilot.1

状态 ACCEPTED CONTENT / PILOT PDF

阅读密度 compact · Visual Profile Candidate, 尚未冻结

基准 commit 8f479deaf660533b2ad82e1f721eb41a363112b6

准备标识 c0ce3381c728be48e0f7cc67

使用方式 目录与书签可跳转；合订版中的核心文档链接跳转到本件对应篇章。独立版在源字节匹配时提供固定 commit 链接，否则指向随附的 source 快照。完整输入以准备记录为准，可能包含未提交变更；正文原有编号不重新连续编号。

G6 v1.1.1 ACCEPTED CONTENT / PILOT PDF

G7 v1.1.1 ACCEPTED CONTENT / PILOT PDF

目录

阅读与版本说明	2
G6 · 内存布局、机器成本与性能测量	5
阅读入口	5
1. 阅读模型	6
2. 对象布局	7
3. 缓存与局部性	10
4. 数据布局: AoS、SoA 与 AoSoA	14
5. 分配、区域与内存资源	17
6. 虚拟内存与地址翻译	21
7. 分支预测与控制流	24
8. 缓存一致性与伪共享	26
9. 测量、剖析与证据	28
10. 统一性能审查	33
11. 跨语言回查	36
12. 术语回查	37
13. 实验与验证	38
14. Final Gate	43
15. Final Gate · 参考答案与常见误判	46
16. 工程原则回查	47
17. G1 ~ G6 的统一系统模型	48
18. 从机器观察转向语言并发模型	49
19. 参考与验证入口	49
G7 · 并发协议与 C++ 内存模型	50
阅读入口	50
1. 阅读模型	51
2. 数据竞争与语言内存模型	51
3. 顺序与同步关系	53
4. 原子性与内存序	54
5. 互斥锁与条件变量	57
6. 原子状态转换与 CAS	60

7. ABA、进展保证与安全回收	62
8. 并发队列的交接协议	64
9. 线程生命周期与停机	66
10. 任务执行、线程池与背压	69
11. 并发架构与写入权限	71
12. 跨层关系图	73
13. 统一并发审查	74
14. 并发反模式	76
15. 工程原则回查	77
16. 实验与验证	78
17. Final Gate	83
18. Final Gate · 参考答案与常见误判	86
19. G0 ~ G7 的统一模型	87
20. 实际代码的跨层审查	88
21. 本章范围与后续阅读	89
22. 参考与验证入口	89

G6 · 内存布局、机器成本与性能测量

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述。当前文件是后续生成的阅读试件，不改变该历史状态，也不表示正式发布获批。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G0 ~ G3/G5；追踪布局、局部性、分配、虚拟内存与跨核成本，以测量检验假设。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G5 · 全系列导航 · 下一章：G7

阅读入口

主线从对象布局（object layout）追到局部性（locality）、分配（allocation）、虚拟内存（virtual memory）、控制流和跨核协调，再用测量（measurement）与性能剖析（profiling）检验假设。结构体数组（AoS，array of structures）与数组组成的结构体（SoA，structure of arrays）只是表示选择，不能预设赢家。先读第 1 ~ 9 节，再用实验和 Gate 检查推理；其余部分供回查。

章节目录

CHAPTER READING PATH · 原文章内导航

- 1. 阅读模型
- 2. 对象布局
- 3. 缓存与局部性
- 4. 数据布局：AoS、SoA 与 AoSoA
- 5. 分配、区域与内存资源
- 6. 虚拟内存与地址翻译
- 7. 分支预测与控制流
- 8. 缓存一致性与伪共享
- 9. 测量、剖析与证据
- 10. 统一性能审查
- 11. 跨语言回查
- 12. 术语回查
- 13. 实验与验证
- 14. Final Gate
- 15. Final Gate · 参考答案与常见误判

- 16. 工程原则回查
- 17. G1 ~ G6 的统一系统模型
- 18. 从机器观察转向语言并发模型
- 19. 参考与验证入口

1. 阅读模型

1.1 文档定位

G6 不再主要学习“C++ 有哪些语法”。它研究的是：一个语义正确的 C++ 程序，最终怎样使用 CPU 和内存，以及这些选择为什么会快或慢。前面几个阶段分别解决：

G1

Object 是否存在？

Lifetime 是否有效？

Access 是否合法？

↓

G2

谁拥有？

谁借用？

谁负责 cleanup？

↓

G3

Value 怎样 copy / move / return？

这些操作真实成本是什么？

↓

G5

哪些差异应该由 compiler 在 compile time specialization？

↓

G6

这些 object / value / abstraction

最终如何映射到 CPU、Cache 和 Memory？

因此 G6 的核心链路是：

C++ Type

↓

Object Representation

↓

Storage Topology

↓

Cache Lines

↓

流程示意 · continued

Virtual Pages / TLB

↓

Allocation Topology

↓

Control Flow

↓

Multi-core Coherence

↓

Measured Performance

1.2 G6 的统一性能模型

性能分析先确定指标，再分解成本来源。计算、数据搬运、访存等待、分配、地址翻译、控制流和跨核协调是审查维度，不是可以直接相加的同量纲耗时公式；它们会重叠执行并互相制约。两个 $O(N)$ 算法可能分别连续扫描和依赖式追指针。表示、访问模式、工作集 (working set)、输入分布和线程拓扑共同决定机器行为。复杂度给出规模趋势，不能替代具体工作负载的观测；G6-M3 会在保持计算结果一致的前提下记录布局差异，而不规定哪一方必须更快。

2. 对象布局

2.1 sizeof(T) 的准确含义

sizeof(T)

表示一个 T object representation 所占的 byte 数，并且必须允许：

T values[N]；

中的相邻 objects 正确排列。所以它并不等于：所有成员 sizeof 简单相加。例如：

[机制片段 · 不承诺独立编译]

```
struct Sample {
    std::uint8_t flags;
    std::uint32_t id;
    float value;
};
```

成员名义大小：

1 + 4 + 4 = 9 bytes

但常见 ABI 下可能得到：

offset 0	flags	1 byte
offset 1	padding	3 bytes
offset 4	id	4 bytes
offset 8	value	4 bytes

sizeof(Sample) = 12

具体数字属于 ABI / implementation，不是 C++ 标准固定值。

2.2 Alignment

`alignof(T)`

表示: `T` object 起始地址必须满足的 `alignment requirement`。例如某平台: `alignof(uint32_t) = 4` 意味着一个正常 `uint32_t` object 的地址通常必须满足: `address % 4 == 0` Alignment 存在的原因可能包括:

CPU load/store requirements、ABI、atomic operations、SIMD/vector instructions、access efficiency。

重要的是: 能够进行 **unaligned access**, 不等于 **alignment** 无意义。

2.3 Internal Padding 与 Tail Padding

Internal Padding

成员之间为了使下一个 member 正确对齐而加入的 bytes。例如:

[机制片段 · 不承诺独立编译]

```
struct S {
    std::uint8_t a;
    std::uint32_t b;
};
```

可能: `[a][pad][pad][pad][ b b b b ]`

Tail Padding

最后一个 member 后面的 padding。例如:

[机制片段 · 不承诺独立编译]

```
struct S {
    std::uint32_t x;
    std::uint8_t y;
};
```

可能: `[x x x x][y][pad][pad][pad]` 这样:

`S values[2];`

中第二个 `S` 仍然从满足 `alignof(S)` 的地址开始。因此: **Tail padding** 的一个核心作用是保证 **array stride** 正确。

2.4 Member Ordering

考虑:

[机制片段 · 不承诺独立编译]

```
struct A {
    std::uint8_t a;
    std::uint64_t b;
    std::uint8_t c;
};

struct B {
    std::uint64_t b;
    std::uint8_t a;
    std::uint8_t c;
};
```

在常见 64-bit ABI 下，A 可能显著大于 B。如果每个 object 差 8 bytes：

```
10,000,000 objects
× 8 bytes
=
80 MB
```

因此 member ordering 会影响：

RAM footprint、cache density、page density、memory bandwidth、TLB reach。

但不能机械执行：“所有 struct 都从大字段排到小字段。”因为 declaration order 同时决定：

construction order、destruction reverse order、ownership teardown dependency、semantic grouping、ABI/layout。

所以：**Member ordering** 是架构设计，不是格式化规则。

2.5 Inline Representation 与 Indirect Representation

Inline

```
std::array<float, 1024>
```

payload 就在 object 内部。因此：

$\text{sizeof}(\text{object}) \approx \text{logical payload footprint}$

Indirect

```
std::vector<float>
```

vector object 本身通常只是类似：

pointer、size、capacity。

而真正的大 payload 位于 dynamic allocation。因此：

$\text{sizeof}(\text{std::vector<float>})$

只能说明：vector wrapper 本身的 representation。不能说明：它拥有的数据总 footprint。同理：

unique_ptr、shared_ptr、string、vector、PImpl。

都必须分析完整 object graph，而不只是 $\text{sizeof}(\text{wrapper})$ 。

2.6 Object Representation ≠ Logical Value

假设：

[机制片段 · 不承诺独立编译]

```
struct S {
    std::uint8_t a;
    std::uint32_t b;
};
```

即使：

```
x.a == y.a && x.b == y.b
```

也不能普遍推出：

```
std::memcmp(&x, &y, sizeof(S)) == 0
```

适合作为 logical equality。因为 object representation 可能包含：

padding、multiple valid representations、pointer representation、floating-point representation details。

所以：

Logical Equality ≠ Byte-for-byte Object Representation Equality

同样：std::is_trivially_copyable_v<T> 不等于 T 可以直接作为 portable wire format。序列化还必须考虑：

endianness、padding、ABI、versioning、schema、type width。

3. 缓存与局部性

3.1 Memory Hierarchy

简化模型：

```
Registers
  ↓
L1 Cache
  ↓
L2 Cache
  ↓
LLC / L3
  ↓
DRAM
```

通常：

```

closer to CPU
→ smaller
→ faster

farther from CPU
→ larger
→ slower

```

因此很多程序不是：CPU arithmetic 不够快。而是：CPU 等不到数据。

3.2 Cache Line

缓存行 (cache line) 是硬件进行缓存填充及一致性管理的重要粒度；其具体大小和层级由处理器决定，C++ 不规定固定为 64 字节。相邻访问可能复用同一行，写入不同对象也可能争用同一一致性单元。用 64 字节推演是一个显式假设，不是本机事实。实验记录操作系统可报告的缓存行和页大小；这些查询也不是完整的缓存拓扑测量。不能仅凭 `sizeof(T)` 和一个假定行宽，就断言实际 miss 数或 false sharing 已经发生。

3.3 Spatial Locality

短时间内访问地址彼此接近的数据。例如：

[机制片段 · 不承诺独立编译]

```

for (const auto& value : values) {
    process(value);
}

```

对于：

`std::vector<T>`

通常：

```

[T][T][T][T][T]
→ → → → →

```

非常适合 spatial locality。

3.4 Temporal Locality

短时间内反复访问同一份数据。例如：

[机制片段 · 不承诺独立编译]

```

state.position += velocity;
use(state.position);
state.position += correction;

```

`state.position` 很可能仍位于：register / cache 因此后续访问更便宜。

3.5 Cache-line Utilization

假设: `cache line = 64 B` 而一个算法每条 `line` 只真正使用: `4 B` 则:

```
useful ratio ≈ 6.25%
```

也就是说: 代码只写了一个 4-byte load, 不代表 `memory subsystem` 只付出了 4 bytes 的成本。这成为 SoA / hot-cold split 的主要动机之一。

3.6 Pointer Chasing

Dense layout: `[T][T][T][T][T]` Pointerized layout:

```
[p][p][p][p]
|  |  |  |
▼  ▼  ▼  ▼
T  T  T  T
```

后者可能引入:

extra pointer load、random heap locations、cache misses、dependent loads、poor hardware prefetch、more pages、larger TLB working set。

所以:

```
std::vector<T>
```

与:

```
std::vector<std::unique_ptr<T>>
```

即使遍历都是: $O(N)$ 实际性能仍可能完全不同。

3.7 Dependent Memory Access

链表:

```
Node0 → Node1 → Node2 → Node3
```

下一地址必须等: `Node0.next` 先从 `memory` 返回。这意味着:

```
load Node0
↓
wait
↓
discover Node1
↓
load Node1
↓
wait
```

多个 `memory misses` 很难充分并行。所以 `pointer chasing` 的一个深层问题是: **Memory-Level Parallelism 较差**。

3.8 Hardware Prefetch

规则访问：

```
X
X+64
X+128
X+192
```

硬件容易预测：很可能还会继续向后读。于是提前请求下一些 cache lines。完全随机访问：

0xA...、0xF...、0x21...。

则难得多。因此：

contiguous、regular-strided。

访问模式通常更硬件友好。

3.9 Working Set

某个时间窗口内真正活跃的数据集合。如果 working set 能较好驻留 cache：

```
cache hits ↑
latency ↓
```

如果远大于 cache：

```
replacement ↑
memory traffic ↑
```

因此：Object footprint 不仅影响 RAM，也决定能有多少 hot objects 同时留在 cache。

3.10 Cache Density

假设：cache line = 64 B 如果：sizeof(T) = 8 一条 line 大约可以容纳：8 objects 如果：sizeof(T) = 32 只能容纳：2 objects 因此 object size 会影响：每次 cache fill 能顺便带进多少 useful objects。

3.11 Latency 与 Bandwidth

Latency

等某个特定数据到达需要多久。Random pointer chasing 容易 latency-bound。

Bandwidth

单位时间能够搬多少 bytes。大型连续 scan 更容易 bandwidth-bound。例如：

[机制片段 · 不承诺独立编译]

```
for (float x : huge_values) {
    sum += x;
}
```

虽然会产生很多 cache misses，但：

sequential、predictable、prefetchable、parallel memory requests。

所以仍可能非常高效。这说明：**Cache miss** 本身不是性能 **verdict**。

3.12 Arithmetic Intensity

粗略定义：

Useful Computation

Bytes Moved

如果：1 add / many bytes 更可能 memory-bound。如果：many FLOPs / few bytes 更可能 compute-bound。因此优化前要问：

- memory-bound?
- compute-bound?
- latency-bound?
- branch-bound?

4. 数据布局：AoS、SoA 与 AoSoA

4.1 AoS — Array of Structures

[机制片段 · 不承诺独立编译]

```
struct SignalSample {
    std::uint64_t timestamp;
    std::uint32_t id;
    float value;
    std::uint8_t valid;
};

std::vector<SignalSample> samples;
```

内存：

[ts id value valid]、[ts id value valid]、[ts id value valid]。

优势：

complete entity contiguous、simple ownership、simple lifetime、simple invariants、natural API。

适合：每次处理一个完整 entity。

4.2 SoA — Structure of Arrays

- timestamps: [ts][ts][ts]
- ids: [id][id][id]
- values: [v][v][v]
- valid: [x][x][x]

适合: 批量扫描某几个字段。例如只计算 `values: [value][value][value][value]` 每条 `cache line` 几乎都装当前 `kernel` 真正需要的数据。

4.3 SoA 的第一收益

不是首先: SIMD。而是: **减少 Overfetch 和无用 data movement**。之后才进一步改善:

vector loads、SIMD、masked processing、column compression。

4.4 SoA 的成本

SoA 将一个 logical row 拆成多个 arrays。于是必须保持:

`timestamps.size()`、`==`、`ids.size()`、`==`、`values.size()`、`==`、`valid.size()`。

产生新的 structural invariant。同时可能增加:

multiple allocations、coordinated reserve/growth、exception-safety complexity、API complexity、borrow invalidation complexity。

所以 SoA 是: 用 representation complexity 换 performance opportunity。

4.5 AoSoA

- Block 0:
- `x[N]`
- `y[N]`
- `z[N]`
- Block 1:
- `x[N]`
- `y[N]`
- `z[N]`

它折中:

```
SoA SIMD friendliness
+
bounded block locality
+
some entity neighborhood
```

常用于:

particles、physics、robotics kernels、HPC。

但实现复杂度高。默认路线应该是:

```

AoS
↓
measured field-wise bottleneck
↓
SoA
↓
clear block/SIMD need
↓
AoSoA

```

4.6 Hot / Cold Split

例如:

[机制片段 · 不承诺独立编译]

```

struct Robot {
    Position position;    // hot
    Velocity velocity;    // hot

    std::string name;     // cold
    DebugInfo debug;      // cold
};

```

physics loop 只需要:

position、velocity。

则拆为:

RobotState、RobotMetadata。

可以同时降低:

cache working set、memory bandwidth、page working set、TLB pressure。

这往往比: 省几 bytes padding 有价值得多。

4.7 Layout 的统一原则

```

Frequently co-accessed data
→ colocate

Independently scanned data
→ separate

Independent high-frequency writers
→ separate cache lines / regions

```

这三条贯穿单线程和多线程 data layout。

4.8 Phase-specific Representation

系统不必只有一种 domain representation。可以：

```
Ingress
→ AoS

Compute
→ SoA

Storage
→ Columnar

External API
→ Row DTO
```

一次 representation transform: $O(N)$ 如果后续：大量重复处理 完全可能值得。因此：**Representation** 可以服务某个 **processing phase**，而不是服务抽象上的“对象纯洁性”。

5. 分配、区域与内存资源

5.1 Allocation 与 Construction

沿用 G1 §1 的对象/存储区分：普通 `new T(args...)` 涉及存储分配 (allocation) 与对象构造 (construction)，匹配的 `delete p`；涉及析构 (destruction) 与存储释放 (deallocation)。本章不重复生命周期规则，只把这四类工作分开计量；资源归还的责任仍由 G2 的所有权合同约定。

5.2 General Heap Allocation 的成本

可能包含：

allocator metadata、size classes、free-list operations、synchronization、cache misses、fragmentation handling、occasionally acquiring more pages。

但：普通 `new` 一般不意味着每次都直接进行 `syscall`。Allocator 通常已经从 OS 获取较大的 memory regions。

5.3 Stack 为什么便宜？

通常的栈帧存储管理利用后进先出的调用结构，可以用栈指针调整批量取得和归还空间，省去一般堆分配器的任意释放搜索与同步。但 **automatic storage duration** 是语言概念，局部对象也可能被放入寄存器或完全消除。“栈便宜”只讨论常见存储管理路径，不包含构造函数、巨大栈帧、触页或栈溢出成本。不能把复杂对象改成局部变量，就据此认定其内部动态分配或清理工作消失。

5.4 Per-item Allocation

例如：

```
std::vector<std::unique_ptr<Detection>>
```

大量 small values 时可能产生：

N allocations、N allocator metadata、N indirections、fragmentation、poor cache locality、more pages、larger TLB working set。

所以如果：

small、value-like、no polymorphism、no address-stability requirement。

通常优先：

```
std::vector<Detection>
```

5.5 Internal / External Fragmentation

Internal Fragmentation

请求：24 B allocator 给：32 B slot 内部没用掉的 8 B。

External Fragmentation

总 free space 足够，但：散成多个小 region 无法满足一块较大的 contiguous request。Fragmentation 不只浪费 RAM，还可能恶化：

page usage、locality、TLB pressure。

5.6 Allocation Optimization Ladder

优先级：

1. Avoid allocation
2. Inline storage
3. reserve + reuse
4. batch allocations
5. Arena
6. Pool
7. Custom allocator/resource
8. Measure again

这是非常重要的顺序。不要：

看到 new
→ 写 memory pool

5.7 reserve + clear

当容量足够时，reserve 加上反复填充与 clear 可以复用 vector 的元素存储，优先于引入自定义分配器。clear 销毁已有元素但保留容量，后续在容量内构造元素不需要重分配这块存储。这个结论不意味着整个过程没有分配或成本：元素自身可能另行分配，析构和重新构造也必须执行。G6-M2 记录指定 PMR 资源收到的请求，并单独核对活对象数；它不代表全进程 allocator profile。

5.8 Arena / Region Allocation

核心：

```
large region

[A][B][C][free.....]
      ↑
    cursor
```

每次 allocation：

align cursor、return cursor、advance cursor。

最终：reset / release whole region 适合：

request lifetime、batch lifetime、parser pass、frame processing、compiler AST。

即：一批 objects 拥有近似共同 lifetime。

5.9 Arena 的真正来源

Arena 快不是因为“特殊内存”。而是因为它限制：individual arbitrary deallocation。因此不需要一般性的：

free-list search、coalescing、per-object reclaim。

再次体现：**Lifetime constraints** 可以换来更简单、更可预测的 **memory management**。

5.10 Arena 与 Object Lifetime

Arena reset 只是在处理：storage。它不会自动正确销毁 placement-new 出来的：

string、vector、file wrapper、mutex。

等 non-trivial objects。因此：

Arena Storage Lifetime $\neq$ C++ Object Lifetime

5.11 Pool

Pool 适合：

many same/similar-size objects + individual arbitrary lifetimes

结构：

```
[used][free][used][free]
      ↑      ↑
    free list
```

allocate: pop free slot free: push slot back

5.12 Arena vs Pool

Arena

- lifetime grouped
- bulk release

Pool

- slots reused independently
- arbitrary per-object lifetime

这是根本区别。

5.13 Stale Generation

Pool:

```
slot 42
generation 7
→ Task A
```

释放再复用:

```
slot 42
generation 8
→ Task B
```

旧 pointer: same address 但已经:

different object、different lifetime、different generation。

所以: **Stable address** 不等于 **stable logical identity**。Generational handle:

[机制片段 · 不承诺独立编译]

```
struct Handle {
    std::uint32_t index;
    std::uint32_t generation;
};
```

可以检测 stale access。

5.14 std::pmr

核心 abstraction:

std::pmr::memory_resource

它把: container/value behavior 和: runtime allocation strategy 进行一定程度解耦。例如:

std::pmr::vector<T>

可以配:

monotonic_buffer_resource、unsynchronized_pool_resource、synchronized_pool_resource、custom resource。

5.15 monotonic_buffer_resource

`std::pmr::monotonic_buffer_resource` 为一组生命周期相近的分配提供单调增长的存储。单独的 `deallocate` 不归还对应块；`release` 或资源销毁统一释放其管理的存储，但不会代替放在其中的对象执行析构。因此顺序应是结束所有相关对象及借用，再释放资源。即使容器已 `clear`，它通常仍持有 `capacity`；这时释放资源后继续使用该容器仍是错误设计。内存资源必须活得比依赖它的容器久。PMR 抽象的是存储分配策略，不是 GC，也不自动提供线程安全。

5.16 Allocation Profile 要看什么？

至少：

allocation count、allocated bytes、size distribution、lifetime distribution、peak live bytes、thread of allocation、thread of free。

尤其：millions of tiny allocations 往往比少数大 allocation 更可疑。

6. 虚拟内存与地址翻译

6.1 Pointer 通常是 Virtual Address

用户态：

`T* p;`

通常首先代表：virtual address。不是直接的 DRAM physical address。每个 process 有自己的：Virtual Address Space

6.2 Pages

Virtual memory 以 page 为主要映射粒度。例如平台可能使用：

4 KiB、16 KiB、large pages、...。

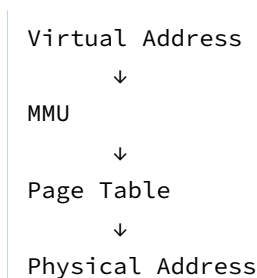
这是 OS / hardware property。Virtual pages：

VPage 100、VPage 101、VPage 102。

可以分别映射到：不同 physical pages 所以：virtual contiguous 不代表 physical contiguous。

6.3 MMU / Page Table

简化：



一个 virtual address 可以概念拆：

Virtual Page Number + Page Offset

映射改变 page number, offset 保持。

6.4 TLB

Translation Lookaside Buffer

缓存:

```
Virtual Page
→
Physical Page
```

的 translation。因此 memory access 同时涉及两套 locality:

Address Translation Locality + Data Cache Locality

6.5 TLB Miss 与 Page Fault

必须严格区分。

TLB Miss

```
translation not in TLB
↓
page-table walk
↓
mapping found
↓
continue
```

page 可能完全正常 resident。

Page Fault

CPU 当前无法完成访问: trap to kernel 可能是:

first anonymous-page touch、copy-on-write、mapped file page、protection fault、illegal address。

因此: Page fault 不等于程序一定出错。

6.6 Allocation ≠ Physical Memory Immediately

可能发生:

```
allocator returns virtual range
↓
not every page physically touched yet
↓
first access
↓
page fault / backing / zeroing
```

所以: malloc/new 10 GB 与: touch 10 GB of actual pages 不是同一件事。

6.7 First Touch

第一次真正访问 large memory:

page faults、page zeroing、mapping work。

可能产生 latency spike。因此 latency-sensitive 系统有时会:

preallocate、pre-touch、reuse。

把成本从: hot phase 搬到: initialization 这叫: **Latency Placement** 而不是凭空消除工作。

6.8 Page Locality / TLB Reach

Dense storage: many useful objects / page 意味着一个 TLB entry 可以服务更多 object accesses。Sparse pointer graph: few useful bytes spread over many pages 则可能:

```
TLB working set ↑
page walks ↑
```

因此 object density 同时影响:

cache density、page density、TLB reach。

6.9 Huge Pages

更大 page:

```
translations needed ↓
TLB reach ↑
```

但可能增加:

internal waste、mapping granularity、fault cost、memory management complexity。

所以只有: translation overhead 已被 profile 证明有意义时才值得考虑。

6.10 mmap

核心不是: “读取文件的另一种 API”。而是: 把 virtual address range 映射到某种 backing object。访问:

mapped[i]

看起来只是普通 pointer access, 但可能隐藏:

TLB miss、page fault、filesystem、storage I/O。

所以: **Pointer syntax** 不揭示 **memory latency class**。

7. 分支预测与控制流

7.1 Branch Prediction

CPU 遇到：

[机制片段 · 不承诺独立编译]

```
if (condition) {  
    foo();  
} else {  
    bar();  
}
```

不能总停下来等 condition 完成。因此：

```
predict branch  
↓  
fetch predicted path  
↓  
execute speculatively  
↓  
verify later
```

7.2 Predictable Branch

例如：99.9% true 通常很容易预测。因此：if 并不天然昂贵。真正可能贵的是：**Branch Misprediction**

7.3 Misprediction

预测错误：

```
wrong-path instructions  
↓  
discard/squash  
↓  
restart correct path
```

成本显著高于单纯 compare。所以 branch 成本取决于：
data distribution、branch history、microarchitecture。

7.4 Branch Cost 是 Runtime-state-dependent

同样：

[机制片段 · 不承诺独立编译]

```
if (valid) {  
    process();  
}
```

数据：99.9% true 和：50/50 random 性能可能完全不同。因此：**Input distribution** 是性能 **specification** 的一部分。

7.5 Source Branchless $\neq$ Machine Branchless

```
count += value >= threshold;
```

源码没有 if，compiler 仍决定具体 lowering。反过来：

[机制片段 · 不承诺独立编译]

```
if (value >= threshold) {
    ++count;
}
```

也可能被编译成 branchless code。所以：查看 optimized machine code，而不是从 C++ syntax 猜。

7.6 Branchless 的 Trade-off

Branchless:

control uncertainty ↓

但可能：

extra work ↑
extra loads ↑
dependency chain ↑

如果 branch 能跳过：

expensive computation、invalid operation、memory load、rare path。

且容易预测，branch 往往更好。

7.7 SIMD 与 Mask

当一条 vector instruction 同时处理多个 lanes：

lane 0 true、lane 1 false、lane 2 true、...。

单一 scalar control-flow branch 很难表达。因此 SIMD 常用：

mask、select、blend、masked operations。

这也是 SoA 更容易 vectorize 的一个原因。

7.8 Fast Path / Slow Path

高概率场景：

short、predictable、hot。

低概率异常：

large diagnostics、error handling、logging。

可以拆：

fast path + cold slow path

这既可能帮助：branch predictability 也可能改善：instruction-cache working set

7.9 `[[likely]]` / `[[unlikely]]`

它们主要是：compiler optimization hints。不是：runtime branch predictor command 也不是 correctness guarantee。应该来自：

real domain distribution、profile、strong invariant。

而不是凭感觉乱加。

7.10 Direct / Indirect Dispatch

Direct call：

```
decode_msg_100();
```

target compile-time known。Function pointer：

```
fn();
```

target runtime known。Virtual dispatch：

```
obj.virtual_method();
```

通常也是 indirect control flow。Indirect dispatch 可能影响：

target prediction、inlining、interprocedural optimization。

但 target 稳定、每次 useful work 足够多时：成本完全可能很小。

7.11 Static Dispatch

Template specialization：

```
process<Camera>();
```

target 更容易在 compile time 决定。优势：

direct call、inline、constant propagation。

代价：

more specializations、larger code、I-cache pressure、compile time。

所以 static dispatch 不是免费胜利。

8. 缓存一致性与伪共享

8.1 Coherence

多核：

Core 0 Cache、 $\updownarrow$ 、coherence、 $\updownarrow$ 、Core 1 Cache。

当同一 memory region 被多个 cores 访问，hardware 必须保持 cache copies 的一致性。重要粒度通常是：cache line / coherence block，而不是 C++ object。

8.2 False Sharing

[机制片段 · 不承诺独立编译]

```
struct Counters {
    std::atomic<std::uint64_t> a;
    std::atomic<std::uint64_t> b;
};
```

Thread 0: writes a Thread 1: writes b 如果:

```
a + b
```

位于同一 cache line, 硬件仍会让 line ownership 在 cores 之间反复转移。这叫: **False Sharing** 即: logically separate、but、physically shared coherence block。

8.3 True Sharing vs False Sharing

True Sharing

多个 threads 真正更新同一个:

```
std::atomic<uint64_t> counter;
```

共享不可避免。

False Sharing

多个 threads 更新不同 values, 只因为物理 colocated 而产生 coherence contention。因此:

```
Race-free
≠
Contention-free
≠
Fast
```

8.4 memory_order_relaxed 不能解决 False Sharing

relaxed 控制的是: C++ memory-ordering contract。但 atomic write 仍然必须获得: 对对应 coherence line 的 writable ownership。因此:

weak ordering + terrible false sharing

完全可能同时存在。

8.5 Shared Immutable Data

多个 cores 只读: immutable Model cache lines 可以保留共享副本。因此: Shared Immutable Data 同时具有:

simpler synchronization + friendly cache-coherence behavior

这也是 immutable snapshot architecture 强大的原因。

8.6 优先减少 Shared Mutation

不要遇到 false sharing 就先 padding。更强的优化顺序：

1. Remove sharing、2. Single writer、3. Per-thread / per-worker state、4. Batch aggregation、5. Cache-line isolation。

例如：

```
shared atomic ++ per event
```

通常远差于：

```
thread-local ++
↓
one aggregate update per batch
```

8.7 Parallel Partition

较好：

```
Thread 0 → contiguous block A
Thread 1 → contiguous block B
```

较差：

```
Thread 0 → even indexes
Thread 1 → odd indexes
```

因为 interleaved writes 很容易让每条 cache line 被多个 cores 同时修改。因此：Parallel partition 应尽可能和 physical ownership region 对齐。

8.8 alignas 与 Interference Size

对齐与填充可以帮助分离频繁写入的状态，但增大 footprint 也可能破坏局部性。std::hardware_destructive_interference_size 是实现提供的干扰间距建议；可用性、值和目标配置必须实测，不能当作所有处理器的真实缓存行枚举。若使用显式 alignas 值，要说明目标假设，并检查数组步长和实际地址关系。单凭两个字段的地址不同不能排除 false sharing；反过来，记录了对齐也不证明该工作负载一定更快。减少共享写、线程本地聚合和批量合并往往比无限加 padding 更重要。

9. 测量、剖析与证据

9.1 性能首先要定义 Metric

可能是：

Latency、Throughput、p99、CPU utilization、Memory footprint、Allocation rate、Startup time、Energy。

“更快”不是完整 requirement。

9.2 Latency vs Throughput

Latency: 一次 operation 多久完成。Throughput: 单位时间完成多少工作。增加 batch size 可能:

```
throughput ↑  
latency ↑  
memory ↑
```

所以必须知道优化目标。

9.3 Tail Latency

平均值可能隐藏极端慢请求。因此系统常看:

p50、p90、p95、p99、p99.9、max。

例如: p99 = 10 ms 表示: 大约 99% observations 不超过 10ms。不是: 最慢 1% 平均值是 10ms。

9.4 Benchmark / Profiler / Tracing

Benchmark

回答: 某个操作有多快?

Profiler

回答: CPU 时间花在哪里?

Tracing

回答: 一条 request/job 的事件时间线是什么? 例如:

```
queued  
↓  
download  
↓  
decompress  
↓  
decode  
↓  
produce
```

它们解决的是不同问题。

9.5 End-to-end First

先看整个系统:

Download 20%、Decompress 40%、Decode 5%、Produce 10%、Waiting 25%。

如果 decode 只占 5%，即使加速很多: end-to-end 也不会提升很多。这就是 Amdahl's Law 的工程直觉。

9.6 Microbenchmark

适合回答：

AoS vs SoA kernel、runtime dispatch vs static dispatch、allocator A vs allocator B。

但必须控制：

setup、allocation、RNG、logging、dead-code elimination、constant folding、cache state、input distribution。

最大的风险是：非常精确地测错问题。

9.7 Optimized Build

生产性能实验必须接近 production：-O2 / -O3 不能以：-O0 作为主要性能结论。因为：

templates、span、iterators、small wrappers。

很多 abstraction 成本只有 optimizer 后才真正体现。

9.8 Dead-code Elimination

如果 benchmark result 从未被使用，compiler 可能删掉整个 computation。所以 benchmark 必须：让结果具有 observable relevance。不要机械用 volatile 解决所有问题。专业 benchmark framework 通常提供专门 optimization barriers。

9.9 Warm-up

第一次运行可能包括：

cold caches、page faults、branch-predictor cold state、dynamic-linking initialization、filesystem cache effects。

所以必须明确：测 cold-start，还是 steady-state？两个都是合法目标，但不能混。

9.10 Dataset Size

Benchmark 应覆盖：

small、medium、large。

使 working set：

fits small cache、fits larger cache、exceeds cache、streams from memory。

否则小 dataset 的结果可能完全不能代表 production。

9.11 Input Distribution

例如：

```
branch:
99% true
vs
50/50 random

hash:
uniform
vs
collision-heavy

string:
SSO short
vs
heap-backed long
```

都会改变结果。所以：Input distribution 是 benchmark specification 的一部分。

9.12 Hardware Counters

可以帮助观察：

cycles、instructions、branch misses、cache misses、TLB misses、loads/stores。

它们解释：为什么慢。但最终是否优化成功仍然由：

end-to-end latency、throughput、tail、memory。

决定。

9.13 Profiler Before Assembly

正确顺序：

```
profile
↓
find hotspot
↓
inspect hotspot assembly
```

Assembly 是：microscope。Profiler 是：map。不要研究一个只占 0.1% CPU 的函数三小时汇编。

9.14 Bottleneck Moves

优化前：

allocator 40%、decode 30%。

优化 allocator 后：

allocator 3%、decode 65%。

说明：新瓶颈浮现。因此必须：

```
profile
→ optimize
→ profile again
```

而不是一次 profile 后一路凭经验优化。

9.15 One Variable at a Time

如果同时：

```
AoS → SoA
heap → arena
branch → branchless
runtime → template
```

最终快 30%，你不知道是谁贡献的。所以尽可能：一次改变一个主要因素。保留：baseline、change、result、interpretation。

9.16 Performance Evidence Ladder

```
Level 0
Intuition

    ↓

Level 1
Timing

    ↓

Level 2
Profiling / Tracing

    ↓

Level 3
Assembly / Hardware Counters

    ↓

Level 4
Production-like / Real System Metrics
```

越往下层越解释 mechanism，越往高层越证明真实系统价值。

10. 统一性能审查

10.1 G6 九层 Review Protocol

任何性能敏感代码都可以按下面顺序检查。

Layer 1 — Object Representation

问：

- sizeof(T)?
- alignof(T)?
- padding?
- inline or indirect?

Layer 2 — Access Pattern

问：

- sequential?
- strided?
- random?
- pointer chasing?

Layer 3 — Working Set

问：

- 多少 bytes?
- 多少 cache lines?
- 多少 pages?
- 是否反复使用?

Layer 4 — Data Layout

问：

- 哪些 fields 是 hot?
- 哪些 fields 总一起访问?
- AoS / SoA?
- Hot/cold split?

Layer 5 — Allocation

问：

- 多少次 allocation?
- size distribution?
- lifetime distribution?

- 是否能 reuse?

Layer 6 — Virtual Memory

问：

- first touch?
- page faults?
- TLB working set?
- dense or sparse pages?

Layer 7 — Control Flow

问：

- branch distribution?
- predictable?
- indirect dispatch?
- branch 能跳过多少工作?

Layer 8 — Multi-core

问：

- 谁写什么?
- shared mutation?
- same cache line?
- single writer possible?
- batching possible?

Layer 9 — Evidence

问：

- 优化目标是什么?
- baseline 是什么?
- profile 证据是什么?
- 改完实际改善多少?

10.2 G6 Performance Smell Catalogue

Smell 1 — Pointerized Small Values

`std::vector<std::unique_ptr<SmallValue>>`

却没有：

polymorphism、stable-address、independent lifetime。

等真实需求。

Smell 2 — Per-item Allocation

[机制片段 · 不承诺独立编译]

```
for (...) {  
    std::make_unique<T>();  
}
```

位于百万级 hot path。

Smell 3 — Cold Data in Hot Struct

```
hot numeric state  
+  
strings  
+  
debug metadata  
+  
rare fields
```

全部混排。

Smell 4 — Random Access on Huge Dataset

却只因为 container 是 vector 就认为 cache-friendly。

Smell 5 — Premature SoA

没有 profile 就把简单 data model 拆成复杂 columns。

Smell 6 — Branchless Cargo Cult

把 highly predictable branch 改成每次都执行昂贵工作。

Smell 7 — Shared Atomic per Event

高频业务事件全部更新同一个 global atomic。

Smell 8 — Per-thread Counters Adjacent

逻辑独立，却共享 cache line。

Smell 9 — Custom Pool Before Reuse

连：

reserve、clear、reuse。

都没试，就自己维护 allocator。

Smell 10 — Huge Template Specialization

只为了消除一个 branch，生成大量近似函数，增加 I-cache/code-size 压力。

Smell 11 — Benchmarking -00

用 debug lowering 得出 production 性能结论。

Smell 12 — Optimizing Without Baseline

只有：“我感觉现在快了。” 没有可重复 benchmark / profile。

11. 跨语言回查

11.1 C++

C++ 的性能优势来自：

explicit representation control、value semantics、contiguous standard containers、compile-time specialization、low-level memory access、RAII。

但相应要求工程师真正理解：

lifetime、aliasing、layout、allocator behavior、cache、memory model。

语言不会自动替你选择最优 representation。

11.2 Zig

Zig 更显式暴露：

allocator、array/slice distinction、layout、comptime。

使：

- where does allocation happen?
- who owns allocator?

通常更容易从 API 看出。但：

cache、TLB、AoS/SoA、false sharing、memory bandwidth。

这些机器规律完全一样。

11.3 Rust

Rust 更强地静态约束：

ownership、borrows、thread safety traits。

但同样不能自动解决：

bad locality、too many allocations、Arc contention、false sharing、poor AoS layout。

因此：**Memory safety** 与 **memory performance** 是不同维度。

11.4 跨语言共同事实

无论：

C++
Zig
Rust

都无法绕开：

cache-line granularity、page granularity、TLB、memory bandwidth、branch predictor、coherence、working set。

这些属于：machine architecture 而不是某种语言哲学。

12. 术语回查

12.1 G6 核心术语表

Term	核心含义
Object Representation	一个 object 在内存中的 byte-level representation
Alignment	object address 必须满足的边界要求
Padding	为 alignment / layout 插入的非成员 value bytes
Tail Padding	object 末尾用于保证相邻 array element 正确对齐的 padding
Cache Line	cache hierarchy 中的重要数据传输/coherence 粒度
Spatial Locality	短时间访问相邻地址
Temporal Locality	短时间重复访问同一数据
Working Set	一段时间内活跃访问的数据集合
Cache Density	单个 cache line 能容纳多少 useful data/object
Pointer Chasing	通过一个 pointer load 的结果决定下一次 memory address
Memory-Level Parallelism	同时存在多个独立 memory requests 的能力
Bandwidth-bound	性能主要由 bytes/sec 限制
Latency-bound	性能主要由单次 memory wait 限制
Arithmetic Intensity	Useful computation / bytes moved
AoS	Array of Structures
SoA	Structure of Arrays
AoSoA	Array of Structures of Arrays
Hot/Cold Split	将高频和低频访问数据分离
Arena	group-lifetime / region allocator
Pool	固定或少数 slot size 的 reusable allocator
Fragmentation	allocation 布局导致的内部/外部空间浪费
PMR	Polymorphic Memory Resource
Virtual Address	process 看到的虚拟地址
Page	virtual-memory mapping 粒度
TLB	virtual→physical translation cache
TLB Miss	address translation cache miss
Page Fault	当前 memory access 需要 kernel/VM 处理
First Touch	首次真正访问某 virtual page
Branch Prediction	CPU 预测 branch control flow
Misprediction	branch outcome 预测错误
Speculative Execution	按预测路径提前执行
Indirect Call	target address 在 runtime 决定的 call
Cache Coherence	多核 caches 对 shared memory 保持一致的机制

Term	核心含义
False Sharing	不同 logical data 因同 cache line 上的写而互相干扰
True Sharing	多线程真正读写同一个 logical state
Benchmark	测量“多快”
Profiling	定位“时间花在哪”
Tracing	描述“事件什么时候发生”
Hardware Counter	CPU 暴露的微架构事件计数
Tail Latency	延迟分布高 percentile 部分
Amdahl's Law	局部优化受未优化部分比例限制的总体收益规律

13. 实验与验证

本章完整实验以 Markdown 中的源文件为准；从仓库根目录运行下列命令。执行器提取文件到新建临时目录，完整命令和原始输出写入结果记录，不修改历史制品。

[命令 · 自动提取、编译及分项记录]

```
python3 c++/learning/verify_handbook.py /usr/bin/clang++
└─ /opt/homebrew/opt/llvm/bin/clang++
```

只有带 h-lab/h-file 标记的完整实验及隔离反例参加本章定向执行；其他机制片段不是已验证的完整实现。改变条件用于理解判据，若未单独运行，不计作新增证据。

13.1 G6-M1 · 对象布局不是语言常量

命题、观察与边界。记录 sizeof、alignof、offsetof，并用静态断言检查数组大小与元素大小的关系；只对语言要求检查，绝不硬编码 Sample 必须为 12 字节。成员顺序的成本意义要结合实际目标及大量对象推理。

[完整实验 · G6-M1 · main.cpp]

```
#include <cstddef>
#include <cstdint>
#include <iostream>
#include <type_traits>

struct Sample { std::uint8_t flags; std::uint32_t id; float value; };
struct Reordered { std::uint32_t id; float value; std::uint8_t flags; };
static_assert(std::is_standard_layout_v<Sample>);
static_assert(sizeof(Sample) % alignof(Sample) == 0);
int main() {
    Sample values[2]{};
    static_assert(sizeof(values) == 2 * sizeof(Sample));
}
```

完整实验 · G6-M1 · main.cpp · continued

```

std::cout << "{\"size\":\"" << sizeof(Sample)
          << "\",\"align\":\"" << alignof(Sample)
          << "\",\"id_offset\":\"" << offsetof(Sample, id)
          << "\",\"value_offset\":\"" << offsetof(Sample, value)
          << "\",\"reordered_size\":\"" << sizeof(Reordered) << "}" << std::endl;
}

```

运行与判据。上述统一命令中的 G6-M1 提取并处理本模块。程序的语义不变量须成立，JSON 数值作为 OBSERVED 保存；不要求固定布局或分配字节数。

13.2 G6-M2 · 资源请求与对象生命周期分别计数

命题、观察与边界。指定 PMR 资源只统计本容器的请求；reserve 后在容量内重复构造与 clear，不增加该资源请求，但仍执行元素析构。不用这个局部计数冒充全进程分配 profile。

[完整实验 · G6-M2 · main.cpp]

```

#include <cstddef>
#include <iostream>
#include <memory_resource>
#include <vector>

struct Counting : std::pmr::memory_resource {
    std::size_t allocations = 0, deallocations = 0, bytes = 0;
    void* do_allocate(std::size_t n, std::size_t a) override {
        void* p = std::pmr::new_delete_resource()->allocate(n, a);
        ++allocations;
        bytes += n;
        return p;
    }
    void do_deallocate(void* p, std::size_t n, std::size_t a) override {
        ++deallocations;
        std::pmr::new_delete_resource()->deallocate(p, n, a);
    }
    bool do_is_equal(const std::pmr::memory_resource& r) const noexcept override {
        return this == &r;
    }
};

struct Item {
    static inline int live = 0;
    Item() { ++live; }
    Item(const Item&) { ++live; }
    Item(Item&&) noexcept { ++live; }
    ~Item() { --live; }
};

int main() {
    Counting resource;
    {

```

完整实验 · G6-M2 · main.cpp · continued

```

    std::pmr::vector<Item> items{&resource};
    items.reserve(64);
    const auto allocated = resource.allocations;
    const auto capacity = items.capacity();
    if (allocated == 0 || Item::live != 0) return 1;
    for (int round = 0; round < 20; ++round) {
        for (int n = 0; n < 64; ++n) items.emplace_back();
        if (Item::live != 64) return 2;
        items.clear();
        if (Item::live != 0 || items.capacity() != capacity ||
            resource.allocations != allocated) return 3;
    }
}
if (Item::live != 0 || resource.allocations != resource.deallocations) return 4;
std::cout << "{\"allocations\": " << resource.allocations
            << ", \"deallocations\": " << resource.deallocations
            << ", \"requested_bytes\": " << resource.bytes << "}" << std::endl;
}

```

运行与判据。上述统一命令中的 G6-M2 提取并处理本模块。程序的语义不变量须成立，JSON 数值作为 OBSERVED 保存；不要求固定布局或分配字节数。

13.3 G6-M3 · 同一求和工作负载的 AoS / SoA 观察

命题、观察与边界。两种布局都读取 key，AoS 另带冷字段。三个规模、一次预热、七轮交替顺序，分配与初始化不计入计时；每次求和必须符合完整期望值。分离编译且不启用 LTO，避免调用者把重复工作折叠；生成汇编供回查。输出 elapsed_ns 只是当前进程观察，不能归因于某个缓存层或当作固定性能排名。

[完整实验 · G6-M3 · kernel.hpp]

```

#pragma once
#include <cstddef>
#include <cstdint>
struct Record { std::uint64_t key; std::uint64_t cold[7]; };
std::uint64_t sum_aos(const Record*, std::size_t);
std::uint64_t sum_soa(const std::uint64_t*, std::size_t);

```


[完整实验 · G6-M3 · kernel.cpp]

```

#include "kernel.hpp"

std::uint64_t sum_aos(const Record* p, std::size_t n) {
    std::uint64_t sum = 0;
    for (std::size_t i = 0; i < n; ++i) sum += p[i].key;
    return sum;
}

std::uint64_t sum_soa(const std::uint64_t* p, std::size_t n) {
    std::uint64_t sum = 0;
    for (std::size_t i = 0; i < n; ++i) sum += p[i];
    return sum;
}

```

[完整实验 · G6-M3 · main.cpp]

```

#include "kernel.hpp"
#include <chrono>
#include <iostream>
#include <vector>

int main() {
    using Clock = std::chrono::steady_clock;
    for (std::size_t n : {1024U, 65536U, 1048576U}) {
        std::vector<Record> aos(n);
        std::vector<std::uint64_t> soa(n);
        std::uint64_t expected = 0;
        for (std::size_t i = 0; i < n; ++i) {
            aos[i].key = soa[i] = i % 251;
            expected += soa[i];
        }
        if (sum_aos(aos.data(), n) != expected ||
            sum_soa(soa.data(), n) != expected) return 1;
        for (int trial = 0; trial < 7; ++trial) {
            for (int position = 0; position < 2; ++position) {
                const bool use_aos = ((trial + position) % 2 == 0);
                const auto begin = Clock::now();
                const auto sum = use_aos ? sum_aos(aos.data(), n)
                                         : sum_soa(soa.data(), n);
                const auto end = Clock::now();
                if (sum != expected) return 2;
                const auto elapsed = std::chrono::duration_cast<
                    std::chrono::nanoseconds>(end - begin).count();
                std::cout << "{\n\n":" << n << ",\ntrial\":" << trial

```

完整实验 · G6-M3 · main.cpp · continued

```
        << ",\layout\":" << (use_aos ? "aos" : "soa")
        << "\",\elapsed_ns\":" << elapsed
        << ",\checksum\":" << sum << "}" << std::endl;
    }
}
}
```

运行与判据。上述统一命令中的 G6-M3 提取并处理本模块。以 -O3 分离编译两个 TU，不启用 LTO；记录 42 条测量及对应校验和，汇编输出属于代码生成观察。**测量解释边界。**G6-M3 的计时含单次函数调用和计时器开销，小规模可能接近分辨率；每轮交替顺序并不消除系统噪声。这里记录的是进程墙钟间隔，不是 CPU 周期、实际 DRAM 流量、缓存/TLB miss 或端到端吞吐。CPU 采样 profile 和硬件计数器本批未采集；汇编不是其替代证据。后续真实优化应另以热点和输入分布为依据。

14. Final Gate

14.1 Object Layout

应能解释：

1. 为什么 `sizeof(struct)` 不等于 `members size sum`?
2. `Alignment` 到底限制什么?
3. `Internal padding` 与 `tail padding` 有什么区别?
4. 为什么 `member reordering` 可以改变大数组 `footprint`?
5. 为什么 `memcmp` 通常不是普通 `struct equality`?
6. 为什么 `sizeof(vector<T>)` 不代表真实 `payload footprint`?

14.2 Cache

应能解释：

1. 什么是 `cache line`?
2. 什么是 `spatial locality`?
3. 什么是 `temporal locality`?
4. 什么叫 `cache-line utilization`?
5. 为什么 `pointer chasing` 不只是多一次 `pointer dereference`?
6. 为什么 `sequential streaming` 可以 `cache miss` 很多却仍然快?
7. `latency-bound` 与 `bandwidth-bound` 区别是什么?
8. `working set` 为什么重要?

14.3 Data Layout

应能解释：

1. `AoS` 和 `SoA` 物理布局区别是什么?
2. 哪种 `workload` 更适合 `AoS`?
3. 哪种 `workload` 更适合 `SoA`?
4. 为什么 `SoA` 往往更容易 `SIMD`?
5. `SoA` 会引入哪些新的 `invariant`?
6. `AoSoA` 在交换什么?
7. 为什么 `hot/cold split` 可能比 `member padding` 优化更重要?
8. 为什么 `domain entity` 不必等价于实际 `C++ struct object`?

14.4 Allocation

应能解释：

1. `new T` 至少包含哪两个阶段？
2. 为什么 `new` 不等于每次 `syscall`？
3. 为什么 `per-item heap allocation` 常常非常昂贵？
4. `stack` 为什么通常 `storage management` 很便宜？
5. `Arena` 的核心 `lifetime constraint` 是什么？
6. `Pool` 和 `Arena` 有什么不同？
7. 为什么 `arena reset` 不能自动替代 `destructor`？
8. 为什么 `stable pool address` 仍然会产生 `stale pointer`？
9. `reserve + clear` 为什么应该优先于 `custom allocator`？
10. `PMR` 真正抽象的是什么？

14.5 Virtual Memory

应能解释：

1. 用户态 `pointer` 为什么通常是 `virtual address`？
2. `virtual contiguous` 与 `physical contiguous` 有什么区别？
3. `page table` 做什么？
4. `TLB` 做什么？
5. `TLB miss` 与 `page fault` 有什么区别？
6. 为什么 `allocation` 与 `physical backing` 不是一回事？
7. `first touch` 为什么会造成 `latency spike`？
8. `dense layout` 为什么也改善 `TLB locality`？
9. `huge pages` 在优化什么？
10. `mmap` 为什么可能让普通 `pointer access` 隐含 `I/O latency`？

14.6 Control Flow

应能解释：

1. `CPU` 为什么需要 `branch prediction`？
2. `branch misprediction` 真正浪费什么？
3. 为什么 `if` 不是天然昂贵？
4. 为什么 `50/50 random branch` 往往难预测？
5. 为什么 `source-level branchless` 不代表 `machine branchless`？
6. `branchless` 为什么可能做更多无用工作？
7. `function pointer` 为什么是 `indirect control flow`？
8. 为什么 `static dispatch` 不一定最终更快？

9. instruction working set 与 code bloat 有什么关系?

14.7 Multi-core

应能解释:

1. cache coherence 解决什么?
2. 什么是 true sharing?
3. 什么是 false sharing?
4. 为什么不同 variables 仍然可能互相拖慢?
5. 为什么 memory_order_relaxed 不解决 false sharing?
6. 为什么 shared immutable data 非常友好?
7. 为什么 per-thread aggregation 往往比 global atomic 强?
8. 为什么 contiguous block partition 比 even/odd interleave 更适合并行写?
9. 为什么 race-free 不代表 fast?

14.8 Measurement

应能解释:

1. latency 和 throughput 有什么区别?
2. p99 表示什么?
3. benchmark / profiler / tracing 分别回答什么?
4. 为什么先做 end-to-end measurement?
5. 什么是 Amdahl's Law 的工程含义?
6. 为什么 microbenchmark 容易测错?
7. 为什么 benchmark 必须使用 optimized build?
8. 为什么 input distribution 是 benchmark specification?
9. 为什么 assembly 应该在 profiling 后看?
10. 为什么优化后必须再次 profile?
11. 为什么 hardware counter 不是最终 verdict?
12. 为什么性能收益必须和 complexity cost 一起评估?

15. Final Gate · 参考答案与常见误判

15.1 Object Layout 与 Cache

Object Layout 1 ~ 6: 成员对齐造成内部空隙, 对象尾部填充保证数组中下一元素对齐, 所以成员大小之和不等于 `sizeof`。对齐限制可用地址; 调整成员次序可能减少 `padding`, 重复到大数组时改变 footprint。普通 `struct` 的 `padding` 与表示差异使 `memcmp` 不能普遍替代逻辑相等; `vector` 本体通常只保存控制状态, 元素存储另计。具体布局以目标 ABI 与 G6-M1 的输出为证, 不能背固定字节数。

Cache 1 ~ 4: 缓存行是传输/一致性的重要粒度; 空间局部性复用邻近地址, 时间局部性复用近期数据, 行利用率描述一行中本次计算真正使用的部分。5 ~ 8: 追指针把下一地址依赖于前次加载, 限制并行访存和预取; 连续流式读取可利用并发请求和带宽, 即使 `miss` 很多也未必慢。latency-bound 等待依赖链, bandwidth-bound 受数据速率限制; 工作集变化会改变驻留行为。单次耗时不能独立判定是哪种瓶颈。

15.2 Data Layout 与 Allocation

Data Layout 1 ~ 3: AoS 把同一对象字段放一起, 适合逐对象使用多字段; SoA 把同字段排成连续数组, 适合跨对象列扫描。4 ~ 6: SoA 的规律访问有利于向量化, 但要维持长度、索引和更新一致性; AoSoA 以块为单位折中行列局部性、SIMD 宽度与尾部处理。7 ~ 8: 热冷拆分避免每次搬运大段不用的数据, 收益可能超过几个 `padding` 字节; 领域实体可映射为多个存储区域, 代价是显式维护关联不变量。G6-M3 只观察一种列扫描, 不为全部工作负载选择 SoA。

Allocation 1 ~ 4: 普通 `new-expression` 结合取得存储与初始化; 通用分配器可从用户态缓存取块, 并非每次系统调用。逐项分配增加管理、碎片和间接访问; 常见栈帧利用后进先出结构批量管理存储, 但对象构造并不免费。5 ~ 8: arena 依赖成组存活/释放, pool 依赖槽位复用; 归还存储不替代析构, 地址复用也可能让旧指针指向错误代次。9 ~ 10: `reserve/reuse` 用简单的容量合同减少请求; PMR 替换分配策略, 不替代对象生命周期与资源依赖顺序。G6-M2 的计数只覆盖指定资源。

15.3 Virtual Memory 与 Control Flow

Virtual Memory 1 ~ 5: 普通用户态地址由页表映射, 虚拟连续不要求物理连续; TLB 缓存翻译, `miss` 可由页表遍历解决, `page fault` 才涉及当前访问需内核处理的异常, 后者也不一定发生磁盘 I/O。6 ~ 10: 分配虚拟区间、取得物理 `backing` 和实际触页不是同一事件; `first touch` 可能引入清零/分配延迟。紧凑布局减少活跃页, `huge pages` 用更大映射范围减少翻译压力, 但有碎片等代价。`mmap` 文件可使首次普通读取触发缺页和 I/O。本批未采集 TLB/缺页硬件事件, 不能从这些机制解释直接推出已经实测。

Control Flow 1 ~ 4: 预测帮助流水线提前取指, 预测错误会丢弃投机工作; 稳定分布的 `if` 可预测, 独立随机的 50/50 分支通常更困难, 但实际相关性由数据决定。5 ~ 6: 编译器可把源码 `if` 转为选择指令, 也可反向生成跳转; 无分支方案可能计算两侧和增加访存。7 ~ 9: 函数指针目标运行时决定, 静态分派可能利于优化, 也可能增加代码体积与指令工作集。先看热点, 再读相关汇编; 不能按源码关键字估算周期。

15.4 Multi-core 与 Measurement

Multi-core 1 ~ 5: 一致性协调核间对共享内存的缓存状态; true sharing 是同一逻辑状态竞争, false sharing 是不同状态碰到同一干扰粒度。relaxed 不取消原子写的所有权协调, 所以不是 false sharing 修复。6 ~ 9: 真正不可变的共享数据减少写失效; 本地聚合减少全局 RMW。连续块分区通常比交错写更易分离写入行, 但要检查边界和实际布局。race-free 只排除一类语言错误, 不保证没有等待、争用或坏局部性。

Measurement 1 ~ 3: 延迟是单次完成时间, 吞吐是单位时间产出; p99 是给定样本分布的百分位, 不是最大值。benchmark 测量工作负载, profile 归因热点, trace 重建事件时间线。4 ~ 6: 端到端指标保证局部热点确实相关; 未优化部分限制总体收益。微基准会受优化消除、输入、热身和计时开销影响, 不能机械外推。

Measurement 7 ~ 12: 测量目标发布配置时使用相应优化选项; 调试构建可用于诊断, 但不能代表优化构建性能。输入分布、规模与顺序都是实验合同。profile 指导汇编阅读, 优化后瓶颈可能转移所以要再测。计数器是可解释信号, 不是因果裁决; 收益还要与维护、内存和构建复杂度比较。未采集的 CPU profile 或硬件 counter 必须登记 NOT RUN, 不用汇编代替。

16. 工程原则回查

16.1 如果半年后只能记住十五条

1. C++ 性能不能从语法直接判断, 必须追到 **object representation**、**access pattern** 和 **machine behavior**。
2. **sizeof(T)** 描述 **object stride**; **alignment** 和 **padding** 会决定大量 **dense objects** 的真实 **footprint**。
3. CPU 的核心成本之一是 **data movement**, 而不是单纯 **arithmetic instruction count**。
4. **Cache line utilization** 决定一次 **memory transfer** 中有多少 **bytes** 真正为当前 **kernel** 服务。
5. **Contiguous dense storage** 的优势来自 **cache locality**、**prefetch**、**memory-level parallelism**、**page locality** 和 **TLB reuse**。
6. **Pointer chasing** 的代价远不止“多一次解引用”: 它会制造 **dependent loads**、**cache misses**、**page spread** 和 **TLB pressure**。
7. **AoS** 与 **SoA** 没有绝对 **winner**; **representation** 必须由 **hot access pattern** 决定。
8. **Hot/cold splitting** 往往比微调几个 **padding bytes** 更值得优先考虑。
9. 最好的 **allocation** 优化首先是不 **allocation**: **inline**、**reserve**、**reuse**、**batch**。
10. **Arena** 用 **grouped lifetime** 换廉价 **allocation**; **Pool** 用固定 **slot topology** 换廉价重复复用; 两者都不会取消 C++ **lifetime rules**。
11. **Virtual address**、**cache**、**TLB**、**page fault** 和 **physical memory** 是不同层次的问题。
12. **Branch** 不天然昂贵; 不可预测的 **control flow** 才可能昂贵, **branchless** 同样可能做更多无用工作。
13. 多线程 **correctness** 和多线程 **performance** 是两个不同问题; **false sharing** 可以让完全 **race-free** 的程序严重退化。

14. 高性能设计优先减少高频跨边界动作：**allocation**、**page fault**、**shared mutation**、**cache-line ownership transfer**、**runtime dispatch**。
15. 任何性能优化都必须经过 **Measure** → **Explain** → **Change** → **Measure Again** 的证据闭环。

17. G1 ~ G6 的统一系统模型

现在我们已经可以用同一个流程分析绝大多数系统 C++。假设看到：

```
std::vector<std::shared_ptr<Record>>
```

不要只看类型名字。按顺序问：

G1 — Object Model

Record / shared_ptr / vector 哪些 objects 存在？

哪些 references/pointers 可能失效？

G2 — Ownership

为什么 Record 需要 shared ownership？

谁延长 lifetime？

G3 — Value Cost

shared_ptr copy 有没有 refcount？

Record 是否可以直接 value-store？

move/copy 成本如何？

G5 — Genericity

哪些行为需要 compile-time specialization？

这里的 template 是否真的必要？

G6 — Machine

多少 allocations？

多少 indirections？

Record 散落多少 pages？

iteration locality 怎样？

shared_ptr control blocks 是否跨线程竞争？

是否存在 false sharing？

真正 profile 热点在哪里？

这份清单把 G1 的有效访问、G2 的共享所有权、G3 的表示成本与本章测量接起来；前几章负责语义，本章只追问这些选择在给定机器上的代价。

18. 从机器观察转向语言并发模型

G6 的成本分析不能替代同步正确性；下一章 G7 §3 以普通数据发布为例，区分硬件一致性与 C++ 可依赖的顺序。机器观察不替代语言顺序；本轮源稿状态与历史本地证据按全书口径分开。

19. 参考与验证入口

全系列导航 · 实验说明 · 本批修订与证据

布局 and 分配结论区分语言合同与目标 ABI；生成汇编的工具选项见 [Clang 用户手册](#)。本批输出不构成处理器性能排名。

G7 · 并发协议与 C++ 内存模型

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述。当前文件是后续生成的阅读试件，不改变该历史状态，也不表示正式发布获批。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G0 ~ G6；从数据竞争、同步和回收到线程停机与并发架构；工程论证不等于形式化证明。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G6 · 全系列导航 · 下一章：G8

阅读入口

本章先区分数据竞争（data race）、原子性（atomicity）和不变量（invariant），再建立顺序与生命周期协议。先发生关系（happens-before, HB）指语言允许依赖的顺序关系，不是墙钟上的先后。首次阅读第 1 ~ 11 节；跨层关系图、审查和反模式用于回查。读写改操作（read-modify-write, RMW）、比较并交换（compare-and-exchange, CAS）都是实现协议的手段，不是设计的出发点。

章节目录

CHAPTER READING PATH · 原文章内导航

- 1. 阅读模型
- 2. 数据竞争与语言内存模型
- 3. 顺序与同步关系
- 4. 原子性与内存序
- 5. 互斥锁与条件变量
- 6. 原子状态转换与 CAS
- 7. ABA、进展保证与安全回收
- 8. 并发队列的交接协议
- 9. 线程生命周期与停机
- 10. 任务执行、线程池与背压
- 11. 并发架构与写入权限
- 12. 跨层关系图
- 13. 统一并发审查
- 14. 并发反模式
- 15. 工程原则回查

- 16. 实验与验证
- 17. Final Gate
- 18. Final Gate · 参考答案与常见误判
- 19. G0 ~ G7 的统一模型
- 20. 实际代码的跨层审查
- 21. 本章范围与后续阅读
- 22. 参考与验证入口

1. 阅读模型

1.1 G7 到底解决什么问题？

G6 讨论缓存、地址翻译和一致性如何影响成本；本章讨论两个线程何时可以合法访问并观察同一状态。这是语言模型问题，不能由“处理器最终会把缓存同步”代替。

工程上应先确定对象存活、写入权限与跨线程交接，再选择 **atomic**、**mutex** 或队列。协议若没有正确的同步和回收关系，某个平台上长期运行正常也不是其合法性的证明。

1.2 G7 的统一链路

本章的推理顺序是：内存位置与冲突访问 → 数据竞争检查 → 原子性及顺序关系 → 对象所有权和生命周期 → 并发协议、进展及容量约束。原子 API 只解决链条中的一部分。

先写清谁拥有数据、谁能修改、哪些字段必须保持一致、何时交接权限及哪些操作需要 HB；之后才选同步原语。队列、线程池和快照都应回到这组问题，而不是从某种“更高级的无锁结构”倒推业务模型。

2. 数据竞争与语言内存模型

2.1 Data Race 是 G7 的第一道门槛

考虑：

[反例片段 · 未定义行为：无同步的冲突访问；不要用于生产代码]

```
int value = 0;
bool ready = false;

// Thread A
value = 42;
ready = true;

// Thread B
while (!ready) {
}

use(value);
```

人类直觉：

```
Thread B 看到了 ready == true
      ↓
Thread A 肯定已经执行了 value = 42
      ↓
所以 value == 42
```

C++ 并不允许这样推理。这里首先存在：**Data Race** 因此：**Undefined Behavior**

2.2 Data Race 的核心条件

数据竞争（data race）的判断对象是潜在并发的冲突操作：修改同一内存位置，或者开始/结束与访问重叠对象的生命周期，都可能形成冲突；至少一方非原子，且没有所需的 happens-before 次序时，会进入未定义行为。只读且生命周期稳定的同一对象通常不构成数据竞争。不同成员是否对应独立内存位置还需留意位域等情况。这个规则不是“读到旧值”的概率模型，更不能用在某次机器运行中恰好没出错来否定。规则：[N4950 intro.races](#)

2.3 read/read 与 read/write

通常：

```
read / read
→ 不冲突
```

而：

```
read / write
write / write
→ conflicting
```

所以：

```
const Config config;
```

在正确 publication 后被多个 threads 只读：是非常自然的模型。真正复杂的是：Shared Mutable State。

2.4 Data Race 不等于 Race Condition

数据竞争（data race）采用 §2.2 的语言判据；竞态条件（race condition）则是更广义的逻辑问题：结果依赖不受控制的交错。前者涉及未定义行为，后者即使只使用原子操作也可能破坏业务不变量。例如：

```
[机制片段 · 不承诺独立编译]

std::atomic<int> balance{100};

if (balance.load() >= 80) {
    balance.fetch_sub(80);
}
```

两个 threads 都可能：

load 100、load 100、subtract、subtract。

最后：-60 所有 atomic operations 本身合法，但业务 invariant 被破坏。所以：

Data-race-free $\neq$ Race-condition-free

2.5 volatile 不是并发同步

C++ volatile 不提供原子性、线程间同步或 happens-before。把共享 ready 标志改成 volatile，不能修复普通 payload 的发布协议。

[反例片段 · 未定义行为风险：volatile 不建立线程同步]

```
volatile bool ready = false;
```

它与 atomic、mutex 承担不同语义责任；应使用适合协议的同步机制，而不是借 volatile 阻止某种优化来模拟线程通信。

3. 顺序与同步关系

3.1 sequenced-before

描述：同一 thread 内 C++ abstract machine 的顺序关系。例如：

[机制片段 · 不承诺独立编译]

```
data = 42;
ready.store(true);
```

有：

```
data = 42
  |
  | sequenced-before
  ▼
ready.store(true)
```

注意：这不是“CPU 一定先把 data 写入 DRAM”。它是：语言级 ordering relation。

3.2 synchronizes-with

跨线程 synchronization primitive 可以建立：**synchronizes-with** 例如 release/acquire：

```
Thread A                                Thread B

release store
  |
  | synchronizes-with
  ▼
                                acquire load
```

前提是：acquire 真正接到了对应的 publication。不是只因为源码中写了两个关键词。

3.3 happens-before

happens-before 是建立合法跨线程访问的重要关系。在本章不使用 consume 的协议中，可以沿线程内 sequenced-before 与跨线程 synchronizes-with 组成传递链。必须逐条说明同步边由哪一次实际观察建立，而不是只标注两端使用了 acquire/release。

如果普通 payload 的冲突访问之间存在所需 HB，且对象生命周期和其余访问也合法，就可以推理这次读写。单条 HB 边不等于整个系统无数据竞争，更不决定所有无关原子操作的全局顺序。

3.4 最重要的一张图

单次发布的关键不是时间先后，而是可组合的关系。生产者写 payload，随后 release-store 标志；消费者的 acquire-load 必须读到该 store（或它引领的适用 release sequence）的值，才能建立 synchronizes-with。结合线程内 sequenced-before，payload 写入 happens-before 消费者读取。

```

producer: payload write --SB--> ready.store(release)
                                     |
                                     SW (load reads this publication)
                                     |
consumer: payload read  <--SB-- ready.load(acquire)

```

G7-D1 仅做一次发布，不重置标志，也不在发布后并发修改 payload。若复用同一槽位，还需消费者完成读取到生产者下一次写入的反向交接；看到 true 不自动授予永久读取许可。

3.5 为什么 data 可以不是 Atomic ?

普通 payload 不必全部改为 atomic：在上述单次发布协议中，写入与读取已由 release/acquire 的 HB 链排序。原子标志承担交接责任，payload 在交接后不再被生产者修改。

这个结论依赖完整协议，不是“有一个 atomic 就保护附近变量”。若另有写者、重置标志后立即覆盖、或对象在读者使用前已销毁，原来的论证必须重做。

4. 原子性与内存序

4.1 Atomicity ≠ Ordering

原子性 (atomicity) 描述某个访问或 RMW 作为不可分割的操作被观察；内存序 (memory ordering) 说明它与其他相关操作建立什么顺序约束。原子对象本身的操作合法，不自动把一组业务操作合成事务。

例如对余额先 load 再 store，两步各自原子却可能丢失并发更新。需要根据不变量选择互斥区或条件 RMW，而不是把每个字段换成 atomic 就结束审查。

4.2 Atomic Operation 三类

Load

```
x.load(order);
```

Store

```
x.store(value, order);
```

Read-Modify-Write

[机制片段 · 不承诺独立编译]

```
x.fetch_add(...);
x.exchange(...);
x.compare_exchange_weak(...);
```

同时有：

read side + write side

4.3 Memory Order 属于 Operation

不是：“这个 atomic variable 是 acquire atomic”而是：

[机制片段 · 不承诺独立编译]

```
x.load(std::memory_order_acquire);

x.store(
    value,
    std::memory_order_release);

x.fetch_add(
    1,
    std::memory_order_relaxed);
```

同一个 atomic object：不同 operations 可以使用不同 memory order。

4.4 memory_order_relaxed

保证：

atomicity + 该 atomic object 自己的 modification order

但不自动建立：surrounding ordinary data 的跨线程 synchronization。典型：

[机制片段 · 不承诺独立编译]

```
processed.fetch_add(
    1,
    std::memory_order_relaxed);
```

适合：

statistics、independent counter、ticket generation。

前提：不依赖它去发布其它 state。

4.5 Modification Order

每个 atomic object 都拥有：对其所有 modifications 的一致总顺序。例如：

```
x:
0
↓
1
↓
2
↓
3
```

所以 relaxed 并不是：“atomic value 本身也完全乱序。”它仍然具有该 object 自己的 atomic coherence。

4.6 memory_order_release

典型角色：**Publish**

[机制片段 · 不承诺独立编译]

```
payload = build_payload();

ready.store(
    true,
    std::memory_order_release);
```

表示：release 之前 sequenced-before 的相关 operations，可以通过合适 acquire 建立跨线程 HB。不要翻译成：flush cache to RAM

4.7 memory_order_acquire

典型：

[机制片段 · 不承诺独立编译]

```
if (ready.load(
    std::memory_order_acquire)) {
    use(payload);
}
```

如果 acquire 观察到对应 release publication，则 release 之前的 writes：HB acquire 之后的 reads。

4.8 Release / Acquire 的核心不是 Cache Flush

release/acquire 是语言排序合同，不是“先把所有缓存刷到 DRAM，再让另一核从 DRAM 重读”。编译器针对目标架构把所需约束映射为适当指令和屏障；硬件也可能直接在缓存之间传递数据。

因此协议正确性在语言关系层证明，成本才在目标代码与机器层测量。二者分开，才能避免因某架构的强顺序或某次汇编看起来简单，就削弱可移植程序的必要关系。

4.9 memory_order_acq_rel

主要用于 RMW：

Acquire previous publication + Publish new state

例如：

[机制片段 · 不承诺独立编译]

```
state.compare_exchange_weak(
    expected,
    desired,
    std::memory_order_acq_rel);
```

但：**RMW 不自动意味着一定需要 `acq_rel`**。如果只需要 `atomicity: relaxed` 可能就够。如果只需要 `acquire: acquire` 也可能够。

4.10 `memory_order_seq_cst`

`memory_order_seq_cst` 除相应 `acquire/release` 作用外，还对 `seq_cst` 操作（包括相应 `fence`）提供符合标准约束的单一总序。它不是把所有非原子访问、混合弱序操作和外部事件都排成一个全局时钟。选择 `seq_cst` 可以简化部分推理，但不能补救对象已销毁、复合不变量被拆散或存在非原子数据竞争的问题。混合内存序时仍需逐条说明同步关系，而不是引用“最强”两个字。规则：N4950 `atomics.order`

4.11 Memory Order 不是性能等级表

内存序不是从 `relaxed` 到 `seq_cst` 的固定性能排行榜。目标架构、具体操作、编译结果和争用都会改变成本；相同关键字在不同场景中不保证相同开销。

选择顺序应是业务不变量、所需 HB、同步协议、足够的内存序，最后才是测量。无证据地降序可能破坏正确性；无分析地全部升到 `seq_cst` 也不能弥补生命周期或进展缺陷。

5. 互斥锁与条件变量

5.1 Mutex 保护的是 Invariant

不是：“锁住一个变量”。例如：

[机制片段 · 不承诺独立编译]

```
struct Account {
    int balance;
    int reserved;
};
```

真正需要保护的是：

`balance >= 0`、`reserved >= 0`、`reserved <= balance`。

也就是：多个字段构成的 `logical invariant`。

5.2 Mutex 提供两件事

Mutual Exclusion

同一时刻：只有一个 `owner` 进入 `critical section`。

Memory Synchronization

一个 `thread`：

```
writes
↓
unlock
```

另一个：

```
successful lock
↓
reads
```

可以建立相应 synchronization / HB。因此 mutex 内保护的数据：

```
int value_;
```

不需要全部改 atomic。

5.3 RAII Locking

默认：

```
std::lock_guard lock{mutex};
```

需要：

unlock/relock、condition_variable、deferred locking。

时使用：

```
std::unique_lock lock{mutex};
```

多个 mutex：

```
std::scoped_lock lock{a, b};
```

优先避免手写：

```
[机制片段 · 不承诺独立编译]

mutex.lock();
...
mutex.unlock();
```

5.4 Critical Section 原则

不是：“越短越好。”而是：覆盖维护 **invariant** 所需的最小完整 **logical transaction**。不能为了缩短锁：

check、unlock、...、lock、update。

把本应 atomic 的业务操作拆开。同时不要无必要地把：

I/O、network、sleep、large computation、callbacks。

放在 hot shared lock 内。

5.5 Deadlock

经典：

- Thread A:
- holds A
- waits B
- Thread B:
- holds B
- waits A

解决方法：

- consistent lock order
- `std::scoped_lock`
- architecture redesign

如果系统出现：

大量 `mutex`、随机组合获取。

应该考虑：是否 `shared-state topology` 本身已经过度复杂。

5.6 Condition Variable 等待的是 Predicate

不是 Notification。正确：

[机制片段 · 不承诺独立编译]

```
cv.wait(lock, [&] {  
    return closed_ || !queue_.empty();  
});
```

真正业务事实：

`queue non-empty`、`or`、`closed`。

`notify_one()` 只是：“state 可能已经改变，请重新检查。”

5.7 Notification 是 Edge，Predicate 是 State

条件变量（`condition variable`）不保存一张可消费的通知清单。正确性依赖受同一互斥协议保护的谓词：生产者持锁改变状态；消费者持锁检查状态，并通过 `wait` 原子地释放锁进入等待。唤醒后重新持锁检查谓词。若通知早于消费者检查，持久状态使它无需等待；若消费者需要等待，锁与 `wait` 的协议避免“检查完但尚未等待”的空窗丢失状态改变。仅把标志改成 `atomic`，再在锁外修改并通知，不自动提供这个保证。G7-D2 把队列、`closed` 和等待条件放在同一把 `mutex` 下。

5.8 Spurious Wakeup

`wait()` 可以在没有目标业务事件时返回。另外多个 waiters 竞争：

A wakes and consumes item、B wakes afterwards、queue empty。

因此：

[机制片段 · 不承诺独立编译]

```
if (!predicate()) {
    cv.wait(lock);
}
```

通常错误。应：

`cv.wait(lock, predicate);`

或：

[机制片段 · 不承诺独立编译]

```
while (!predicate()) {
    cv.wait(lock);
}
```

6. 原子状态转换与 CAS

6.1 为什么 load + store 不等于 Atomic Transition ?

[机制片段 · 不承诺独立编译]

```
if (state.load() == Ready) {
    state.store(Running);
}
```

两个 individually atomic operations，仍然可能：

Thread A load Ready、Thread B load Ready、Thread A store Running、Thread B store Running。

两个 threads 都认为自己抢到了 transition。

6.2 CAS

[机制片段 · 不承诺独立编译]

```
value.compare_exchange_strong(
    expected,
    desired);
```

语义：

- if `current == expected`:
- `current = desired`
- return true
- else:

- expected = current
- return false

关键：CAS 把 check + update 合成一个 atomic RMW。

6.3 expected 是 In/Out Parameter

成功：

atomic = desired、expected unchanged、return true。

失败：

atomic unchanged、expected = actual observed value、return false。

这就是为什么 CAS loop 不需要每次重新 load。

6.4 CAS Loop

```

load current
↓
compute desired
↓
CAS
├ success → done
└ failure
    ↓
    current updated
    ↓
    retry
  
```

这是：**Optimistic Concurrency** 观察:current state 尝试:conditional commit 冲突:retry

6.5 Weak vs Strong

compare_exchange_weak 允许伪失败，适合本来就要重试的循环；compare_exchange_strong 不允许这种伪失败，适合需要解释单次失败含义的用法。两者仍须处理值确实改变的情况。

weak/strong 描述 CAS 的失败模型，relaxed/acquire/release 描述排序与同步，不能互相替代。循环中还要根据更新后的 expected 重算 desired，并检查进展和重复副作用。

6.6 Success / Failure Ordering

双内存序 CAS 必须分别解释成功的 read-modify-write 和失败的 load。失败不执行写入，因此 failure order 不得是 release 或 acq_rel；应按失败后是否读取发布数据选择允许的顺序，不能把成功侧的 release 要求机械复制过来。

[机制片段 · 不承诺独立编译]

```
while (!state.compare_exchange_weak(expected, desired,
                                     std::memory_order_acq_rel,
                                     std::memory_order_acquire)) {
    desired = next(expected); // 依据更新后的 expected 重算。
}
```

`next` 必须与实际状态转换合同匹配，循环内也不能不经处理地重复外部副作用。`weak CAS` 的伪失败与 `memory order` 的强弱是两个问题。

7. ABA、进展保证与安全回收

7.1 CAS 只比较 Current Value

CAS 比较当前值表示 (value representation)，不是调用用户定义的 `operator==`，也不记录对象经历过的历史。指针或索引可能从 A 变为 B 再回到相同表示 A；CAS 看到相等仍无法判断它是否属于同一逻辑代次，这就是 ABA。需要历史身份时，可以把 `generation/tag` 纳入被比较状态；还要考虑宽度、回绕和原子实现条件。`tag` 并不保护已经取得的指针所指对象，内存回收协议仍是独立责任。

7.2 ABA 不是 Use-after-free

ABA：state/version/history 问题。Reclamation：object lifetime 问题。两者经常同时出现，但必须分开。

7.3 Unlinked ≠ Reclaimable

一个 node：

```
Linked
↓
Unlinked
↓
Retired
↓
Reclaimable
↓
Destroyed
```

从 lock-free structure 中删除：只解决 Logical Membership。不代表：没有 reader 仍持有 pointer。

7.4 Pointer Acquisition ≠ Safe Dereference

并发：

```
Node* p = head.load();
```

只得到：pointer value。并没有自动保证：`*p` object lifetime 仍然 active。另一个 thread 可能：`remove`、`delete`。

发生在：`load p` 和：`p->next` 之间。

7.5 Tagged Pointer / Generation

把: pointer 扩展成: (pointer, generation) 例如:

```
(A, 41)
→
(B, 42)
→
(A, 43)
```

旧: (A, 41) 不再等于: (A, 43) 可以检测一类 ABA。但: 不自动解决 pointee lifetime。

7.6 Reclamation Strategies

Reference Counting

```
reader holds ownership
→ node cannot die
```

简单组合, 代价:

atomic refcount、control block、coherence traffic。

Hazard Pointer

reader 显式发布: I currently protect Node A reclaimer:

```
A appears in hazard set
→ cannot free
```

流程:

```
load pointer
↓
publish hazard
↓
revalidate source
↓
dereference
↓
clear hazard
```

Epoch-based Reclamation

reader 表示: I am active in epoch E retired node: 等所有可能仍在旧 epoch 的 readers 离开后再 reclaim。

RCU / QSBR

```
build new generation
↓
publish
↓
wait grace period
↓
reclaim old generation
```

特别适合:

read-heavy、write-rare。

系统。

7.7 Lock-free 不是性能等级

Lock-free 是：Progress Guarantee。不是：“一定比 mutex 快”。CAS contention 可能产生：

retry storm、cache-line ping-pong。

一个低 contention mutex 可能更快、更简单。

7.8 Progress Guarantees

粗略：

Wait-free、 $\Rightarrow$ 、Lock-free、 $\Rightarrow$ 、Obstruction-free。

Lock-free

系统整体保证：持续有某个 operation 取得进展。不保证每个 thread 都不饿死。

Wait-free

更强：每个 operation 都在有限步骤内完成。

7.9 atomic<T>::is_lock_free() 不是算法证明

它只回答：这个 atomic object 的 implementation 是否 lock-free。不代表：整个 data structure / allocator / reclamation / operation path 是 lock-free。

8. 并发队列的交接协议

8.1 Queue 首先按 Topology 分类

Queue	Producer	Consumer
SPSC	1	1
MPSC	N	1
SPMC	1	N
MPMC	N	N

Topology 直接决定：

- 谁写 enqueue cursor?
- 谁写 dequeue cursor?
- 哪里需要 CAS?
- 哪里可能 single-writer?

8.2 SPSC 的关键简化

- Producer:
- only writer of tail
- Consumer:
- only writer of head

因此：不需要 CAS 去争抢 cursor ownership。只需正确 publication。

8.3 SPSC 的两条 HB Edge

Producer → Consumer

```
construct slot
↓
tail.store(release)
↓
tail.load(acquire)
↓
consume slot
```

因此：

slot write、HB、slot read。

Consumer → Producer

```
finish consuming
↓
head.store(release)
↓
head.load(acquire)
↓
reuse slot
```

因此：

old generation use、HB、new generation construction。

这两条就是 SPSC correctness 骨架。

8.4 MPSC 的关键新问题

多个 producers 可以：reserve slots 但 reservation completion 可以 out-of-order：

```
P0 reserves slot 10
pause

P1 reserves slot 11
writes + publishes
```

于是：

reservation frontier $\neq$ ready frontier

简单 global tail 不够。

8.5 MPMC 为什么需要 Per-slot Generation

常见有界 MPMC 环形队列用每槽 sequence/generation 区分空闲、已占位、已发布及可复用的代次。多生产者的 reservation 不等于元素构造和发布已经完成；消费者必须按设计好的槽状态取得元素。这不是所有 MPMC 算法都必须采用同一种布局的定理。序号回绕、失败路径、对象构造/析构及消费者回收都需要单独证明。本章解释协议问题，不把一个 per-slot 数字当作完整队列正确性证明。

8.6 Bounded Queue = Backpressure

如果 producer rate:

、consumer rate。

无界 queue:

memory ↑
latency ↑

bounded queue: full 显式告诉上游：系统已达到当前 processing capacity。Full 是：正常系统状态。不是异常。

8.7 Queue 是 Ownership Boundary

成功:

```
queue.push(std::move(job));
```

自然语义:

```
Producer owns Job
↓
Queue owns Job
↓
Consumer owns Job
```

因此跨线程 queue 中优先:

owned value、move-only handle、lease、unique ownership。

而不是未经约束的 borrowed pointer。

9. 线程生命周期与停机

9.1 std::thread

如果 destructor 时: joinable == true 会: std::terminate()。所以 thread lifetime 必须显式解决:

join、or、detach。

9.2 detach() 不解决 Lifetime

它只是：解除 `std::thread` object 与执行线程的 join relationship。不会解决：

`thread still accesses this`、`thread still accesses queue`、`thread still accesses stack object`。

所以 `detach` 常常只是：隐藏 ownership 问题。

9.3 std::jthread

核心：

RAII join + cooperative stop

Destructor 大体：

```
request_stop
↓
join
```

但：stop 是 request，不是 kill。Worker 必须 cooperate。

9.4 stop_token

表示：观察 cancellation request 的 capability。不是：pause/resume state 而是 monotonic：

```
not requested
↓
requested
```

9.5 Cancellation 必须唤醒 Blocked Thread

合作取消必须同时解决请求可见、阻塞可唤醒和退出后可 join。`request_stop()` 只提出请求，不会强制终止任意 wait、I/O 或持锁操作。

一种方案是在同一 mutex 下设置 closed 等终止谓词并通知所有等待方；另一种是使用 `condition_variable_any` 提供的 stop-token 等待重载。仅注册一个在锁外调用 `notify_all` 的 stop callback，仍可能遗漏检查与等待之间的竞争窗口。普通条件变量实验 G7-D2 使用显式 close；它不冒充 stop-token 重载的验证。规则：N4950 [condition_variable_any](#)

9.6 Cancellation Point 必须保持 Invariant

不能：

```
update half of shared state
↓
see stop
↓
return
```

如果状态因此不一致。Cancellation 和 Exception Safety 有同样结构：

```

prepare
↓
safe cancellation point
↓
commit

```

9.7 Member Destruction Order

成员按声明的逆序析构，因此让工作线程成员最后声明，可以使它先于其访问的依赖成员析构。但声明顺序只是前提之一：线程必须收到可执行的停止协议，被阻塞时能唤醒，且 `join` 不与被持有的锁形成死锁。`jthread` 析构在仍为 `joinable` 时请求停止并 `join`，却不能替代用户协议。若工作函数不响应停止、继续等待外部资源，或者回调重新进入已在析构的 `owner`，仅调整成员顺序并不能解决问题。

9.8 Shutdown 基本顺序

1. 停止接收新工作。
2. 发布关闭或停止状态。
3. 唤醒阻塞线程。
4. 按合同排空已接收工作，或取消并处理未完成工作。
5. 等待线程函数退出。
6. 完成 `join`。
7. 销毁共享状态及其资源。

具体依赖可以要求更细的步骤，但不得在仍有访问者时先销毁依赖。

9.9 Drain vs Abort

Drain

```

reject new work
↓
process already accepted work
↓
exit

```

Abort

```

reject new work
↓
request cancellation
↓
discard/cancel according to contract
↓
exit

```

两者绝不能用一个模糊：

`stop();`

让 `caller` 猜。

10. 任务执行、线程池与背压

10.1 Thread ≠ Task

Thread: execution resource。Task: unit of work。Thread pool 的本质:

```
many tasks
↓
bounded worker threads
```

把: logical concurrency 和: physical parallelism 解耦。

10.2 C++23 Task Representation

非常适合:

[机制片段 · 不承诺独立编译]

```
using Task =
    std::move_only_function<void()>;
```

因为 Task 经常捕获:

unique_ptr、socket、buffer lease、move-only handle。

这和 ownership transfer 模型一致:

```
Submitter
→ Queue
→ Worker
```

10.3 Worker 不能持 Queue Lock 执行 Task

正确:

```
lock queue
↓
move task out
↓
unlock
↓
execute task
```

Queue mutex 只保护: queue invariant。Task execution 已经变成: worker-local ownership。

10.4 Pool 必须考虑两种 Bound

Worker count 限制: physical parallelism。Queue capacity 限制: backlog。只有 bounded workers、unbounded queue: overload 仍然可以把 memory/latency 推爆。

10.5 CPU-bound 与 I/O-bound Pool

CPU-bound:

```
worker count
≈ available compute resources
```

从此附近开始 benchmark。I/O-bound: threads 可能大量 blocked, worker count 可能更大。但更成熟可能使用: async I/O 避免大量 blocked OS threads。

10.6 Resource Isolation

不要把:

CPU tasks、blocking database calls、fsync、network waits、background cleanup。

全部塞进一个 global pool。否则: blocking work 会 starve CPU work。典型:

CPU pool、blocking pool、async I/O runtime。

分离。

10.7 Thread Pool Starvation Deadlock

Pool 只有一个 worker:

```
Task A runs
↓
submit Task B to same pool
↓
A waits future B
↓
B waits for a worker
↓
the only worker is occupied by A
```

没有 mutex cycle, 仍然 deadlock。这是: execution-capacity dependency cycle。

10.8 Bounded Submit 也能 Deadlock

多个 workers:

```
each running task
↓
blocking submit child task
```

queue full。所有 workers: wait queue capacity 但 queue capacity 只能通过: worker consuming tasks 来释放。于是:

```
Workers
↓ wait for
Queue Capacity
↓ requires
Workers
```

形成 cycle。Backpressure 必须结合：resource graph 分析。

10.9 Global Queue vs Work Stealing

Global MPMC：

simple、good natural balancing、central contention。

Per-worker queue：

locality、less global contention。

空闲 worker：steals Work stealing 本质：common case local ownership, rare case cross-worker coordination。

11. 并发架构与写入权限

11.1 并发设计最重要的问题：有几个 Writer？

不是：有几个线程。而是：

- 每一份 mutable state
- 有几个 writer？

复杂度通常随着：multiple writers 急剧上升。

11.2 Reasoning Complexity Ladder

粗略：

```
Thread-local mutable
    ↓
Single-writer mutable
    ↓
Immutable shared
    ↓
Ownership transfer
    ↓
Mutex-protected shared mutable
    ↓
Atomic shared mutable
    ↓
Multi-object lock-free shared mutable
```

不是性能排名。而是：reasoning complexity。

11.3 Privatize First

例如 metrics：差：

```
32 workers
→ one global atomic counter
```

更好：

```
each worker
→ own counter
↓
periodic aggregation
```

这可以显著减少：

coherence traffic、atomic contention。

所以：**Privatize first, synchronize later.**

11.4 Single Writer Principle

单写者（single writer）使状态转换集中在一个序列中，减少多写者竞争和不变量证明分支。线程本地状态、按实体分片及消息传递，都可以用来构造这种所有权边界。单写者不意味着其他线程可无同步读取同一可变对象。读者仍需快照、锁、消息回复或经过论证的发布协议；消息本身也要说明转移后谁可以继续访问底层数据。

11.5 Message Passing 的真正价值

不是：“不用 shared memory。” Queue 本身仍然共享。真正变化的是：**Mutation Authority** 从：

```
Many writers
→ same state
```

变成：

```
Many producers
→ queue
→ one owner
→ state
```

11.6 Sharding

如果 single writer 成为 throughput ceiling：

```
key
↓
shard
↓
single owner
```

例如：

```
EntityId → shard 0
EntityId → shard 1
...
```

得到：

- within shard:
- single writer
- across shards:

- parallelism

11.7 Shard Key = Ownership Key

Sharding 的本质不是：“分几个 queue”。而是：哪个 key 决定谁拥有写入权限 (mutation authority)? 例如：

VehicleId、AccountId、PartitionId、ConnectionId。

选错 shard key: cross-shard coordination 会非常多。

11.8 Immutable Snapshot

不可变快照 (immutable snapshot) 把读者需要的一组字段作为一致版本发布，适合读多写少且允许读到稍旧版本的场景。读者取得快照所有权后不需要逐字段争锁，但发布与旧版本回收仍需协议。

```
std::atomic<std::shared_ptr<const Model>> current;
```

const Model 只约束这条访问路径，不自动证明深层对象不可变；若写者仍持有可变别名并同时修改内容，快照承诺便被破坏。shared_ptr 保证的是所管理对象的生命周期，不是所有关联状态的线程安全。

11.9 Snapshot Consistency vs Freshness

Reader 可能持有：V1 而 current 已经：V2 这可能是：stale but internally consistent。是否允许由 product/system contract 决定。不要把：latest 和：consistent generation 混为一谈。

11.10 Old Generation 的 Lifetime

not current $\neq$ safe to destroy

Old reader 可能还在使用 V1。因此仍需要：

shared_ptr、epoch、RCU。

等 generation reclamation。这和 lock-free node reclamation 完全同构。

12. 跨层关系图

12.1 Ownership Graph

所有权图标出对象的 owner、borrower、转移点与销毁者。同步边并不自动延长存活时间，所以线程开始工作之前，就要知道谁保证它访问的对象仍然存在。

12.2 Synchronization Graph

同步图标出每条 HB 的来源：读到发布的 acquire、相应 mutex 的解锁/加锁、线程完成后的 join 或已论证的队列交接。不能只画“线程 A $\rightarrow$ 线程 B”而省略触发该边的实际操作。

12.3 Mutation Graph

写入权限图（mutation authority graph）为每份状态列出全部写者、允许的读者与权限交接。多写者是重点审查位置，单写者也必须解释跨线程读取如何安全发生。

12.4 Progress Graph

进展图描述谁等待谁、谁有能力使谁继续，检查死锁、线程池饥饿和背压环。只有容量限制而没有释放容量的可执行路径，系统仍可能停住。

12.5 Lifetime / Reclamation Graph

生命周期与回收图区分逻辑移除和实际销毁。标出最后可能访问该对象的读者，以及证明这些访问已经结束的条件；不要把从容器 `erase` 当作跨线程的回收屏障。

12.6 Cache Ownership Graph

缓存所有权图追踪哪些核心高频写哪些物理邻近数据。它用于 G6 的争用/伪共享分析；`race-free` 仅是正确性前提，不等于一致性流量低。

12.7 Queue / Capacity Graph

容量图标出压力积累位置、满队列后的行为以及释放容量依赖谁。与进展图联读，可以发现 `worker` 全部阻塞在 `submit`、却无人能继续消费的闭环。

13. 统一并发审查

任何并发问题，建议按以下顺序。

Layer 1 — State

共享的 `state` 到底是什么？

Layer 2 — Ownership

- 谁拥有它？
- 谁控制 `lifetime`？

Layer 3 — Writers

- 有几个 `writer`？
- 能不能降到 1？

Layer 4 — Conflict

- `read/read`？
- `read/write`？
- `write/write`？

Layer 5 — Atomicity

- ordinary?
- atomic?
- compound invariant?

Layer 6 — Ordering

HB edge 从哪里来？

Layer 7 — Synchronization

选择：

- mutex?
- CV?
- atomic?
- CAS?
- queue?
- join?

Layer 8 — Lifetime

reader 持有 pointer 时 object 能不能死？

Layer 9 — Progress

- blocking?
- lock-free?
- wait-free?
- 谁会饿死？

Layer 10 — Backpressure

full / overload 时怎么办？

Layer 11 — Shutdown

- drain?
- abort?
- wake?
- join?

Layer 12 — Architecture

最后问：

- 这份 shared mutable state
- 真的需要共享修改吗？

这是最重要的一层。

14. 并发反模式

14.1 Smell 1 — Global Mutable State

```
all threads  
→ one global map/state
```

然后不断增加：

mutex、atomics、flags。

14.2 Smell 2 — Atomic Every Field

[机制片段 · 不承诺独立编译]

```
struct State {  
    std::atomic<int> a;  
    std::atomic<int> b;  
    std::atomic<int> c;  
};
```

但没人定义：multi-field consistency。

14.3 Smell 3 — Relaxed Cargo Cult

看到 atomic：

memory_order_relaxed

只因为：“更快”。却画不出 HB proof。

14.4 Smell 4 — Lock-free Cargo Cult

存在 mutex：第一反应就是 CAS。没有 profile，也没有 progress requirement。

14.5 Smell 5 — Detached Thread

```
std::thread{...}.detach();
```

但 thread lifetime、dependencies、shutdown 全不明确。

14.6 Smell 6 — Unbounded Queue

用：“不阻塞 producer”掩盖 processing capacity mismatch。

14.7 Smell 7 — Worker Waits Same Pool

Pool worker：

```
submit child  
↓  
wait child
```

没有分析 execution-capacity cycle。

14.8 Smell 8 — Notification as State

I got notify、therefore condition true。

错误。Predicate 才是 truth。

14.9 Smell 9 — Remove Then Delete

Lock-free structure：

```
CAS remove node
↓
delete immediately
```

却没有 reclamation proof。

14.10 Smell 10 — Shared shared_ptr<MutableT> Everywhere

共享拥有可以延长生命周期，却没有确定写入权限（mutation authority）。

14.11 Smell 11 — All Workers Touch All Entities

随机 dispatch 所有 state，造成：

locks、cache migration、poor affinity。

14.12 Smell 12 — Shutdown as One Bool

bool running；

却没有定义：

- accepting?
- draining?
- aborting?
- stopped?

15. 工程原则回查

如果半年后只能保留十五条：

1. 在讨论线程“看见什么”之前，先证明程序没有 **Data Race**。
2. **Happens-before** 是 C++ 并发推理的核心关系，不是 **wall-clock** 时间顺序。
3. **Atomicity**、**memory ordering**、**logical transaction** 是三个不同概念。
4. **Release/Acquire** 的本质是建立 **publication** → **consumption** 的 **synchronization relation**，不是刷新 **cache**。
5. **Mutex** 保护的是 **shared-state invariant**，而不是某个孤立变量。
6. **Condition variable** 等待的是 **predicate**；**notification** 只是唤醒提示。

7. **Atomic load + store** 不构成 **atomic state transition**；**CAS** 才能表达 **conditional commit**。
8. **CAS success** 不解决 **pointer lifetime**；**lock-free structure** 必须单独证明 **memory reclamation**。
9. **ABA** 的本质是 **current value** 恢复相同，但 **logical history/generation** 已改变。
10. **Lock-free** 是 **progress guarantee**，不是性能排名。
11. **SPSC/MPSC/SPMC/MPMC** 的区别本质上是 **writer/claimer ownership topology** 不同。
12. **Thread** 本身具有 **lifetime**；**shutdown** 必须遵循 **stop accepting → wake → drain/abort → exit → join → destroy**。
13. **Bounded queue/backpressure** 是系统稳定性 **contract**，不是容器细节。
14. 并发复杂度主要来自 **Shared Mutable State + Multiple Writers**，因此优先 **thread-local**、**single-writer**、**sharding**、**immutable snapshot** 和 **ownership transfer**。
15. 优秀并发架构的目标不是使用更聪明的 **synchronization**，而是让绝大多数业务 **state** 根本不需要 **concurrent mutation**。

16. 实验与验证

本章完整实验以 Markdown 中的源文件为准；从仓库根目录运行下列命令。执行器提取文件到新建临时目录，完整命令和原始输出写入结果记录，不修改历史制品。

[命令 · 自动提取、编译及分项记录]

```
python3 c++/learning/verify_handbook.py /usr/bin/clang++
└─ /opt/homebrew/opt/llvm/bin/clang++
```

只有带 **h-lab/h-file** 标记的完整实验及隔离反例参加本章定向执行；其他机制片段不是已验证的完整实现。改变条件用于理解判据，若未单独运行，不计作新增证据。

16.1 G7-D1 · 单次发布的不变量与动态观察

命题、观察与边界。 每轮创建新的 **payload** 和 **ready**，写者只写一次；读者只在 **acquire** 观察到发布后读取。写入 **SB release**、**release SW acquire**、**acquire SB** 读取给出 **HB 链**。**join** 完成后才销毁状态；有限 **stress** 与 **TSan** 无诊断仪支持本次路径，不证明所有调度、公平性或复用协议。

[完整实验 · G7-D1 · main.cpp]

```
#include <atomic>
#include <iostream>
#include <thread>

int main() {
    for (int round = 0; round < 200; ++round) {
        int payload = 0;
        std::atomic<bool> ready{false};
        bool correct = false;
        std::thread consumer([&] {
```

完整实验 · G7-D1 · main.cpp · continued

```

        while (!ready.load(std::memory_order_acquire))
            std::this_thread::yield();
        correct = (payload == 42);
    });
    std::thread producer([&] {
        payload = 42;
        ready.store(true, std::memory_order_release);
    });
    producer.join();
    consumer.join();
    if (!correct) return 1;
}
std::cout << "publication=200" << std::endl;
}

```

运行与判据。上述统一命令中的 G7-D1 提取并处理本模块。先运行普通构建，再在 TSan 可用时运行插桩构建；需要指定输出和正常退出，超时为失败而非停机成功。

16.2 G7-D2 · 有界通道：排空、拒绝与停机

命题、观察与边界。队列和 closed 由同一 mutex 保护；push 插入与 close 置位是各自线性化点。pop 在 closed 且空时结束，否则按 FIFO 取出；close 后不接受新值，已接受值仍排空。notify 只负责唤醒，谓词决定动作。实验检查完整 FIFO、空通道关闭、满通道关闭及重复 close；等待中的计数器在同锁下登记，保证关闭确实发生在等待路径，不用 sleep 猜时序。

[完整实验 · G7-D2 · main.cpp]

```

#include <condition_variable>
#include <deque>
#include <iostream>
#include <mutex>
#include <optional>
#include <thread>

class Channel {
    std::mutex mutex_;
    std::condition_variable changed_;
    std::deque<int> queue_;
    bool closed_ = false;
    unsigned waiting_ = 0;
public:
    bool push(int value) {
        std::unique_lock lock{mutex_};
        while (!closed_ && queue_.size() == 4) {
            ++waiting_; changed_.notify_all();
            changed_.wait(lock);
            --waiting_;
        }
    }
}

```

完整实验 · G7-D2 · main.cpp · continued

```

        if (closed_) return false;
        queue_.push_back(value);
        changed_.notify_all();
        return true;
    }

    std::optional<int> pop() {
        std::unique_lock lock{mutex_};
        while (!closed_ && queue_.empty()) {
            ++waiting_; changed_.notify_all();
            changed_.wait(lock);
            --waiting_;
        }
        if (queue_.empty()) return std::nullopt;
        int value = queue_.front();
        queue_.pop_front();
        changed_.notify_all();
        return value;
    }

    void close() {
        std::lock_guard lock{mutex_};
        closed_ = true;
        changed_.notify_all();
    }

    void wait_until_blocked() { // Test observation, not a production API.
        std::unique_lock lock{mutex_};
        changed_.wait(lock, [&] { return waiting_ != 0; });
    }
};

int main() {
    for (int round = 0; round < 50; ++round) {
        Channel channel;
        bool producer_ok = true, consumer_ok = true;
        int count = 0;
        std::thread reader([&] {
            while (auto value = channel.pop()) {
                if (*value != count) consumer_ok = false;
                ++count;
            }
        });
        std::thread writer([&] {
            for (int i = 0; i < 1000; ++i)
                if (!channel.push(i)) producer_ok = false;
            channel.close();
        });
        writer.join();
        reader.join();
        channel.close();
    }
}

```


完整实验 · G7-D2 · main.cpp · continued

```

        if (!producer_ok || !consumer_ok || count != 1000 ||
            channel.push(1001) || channel.pop()) return 1;
    }
    Channel empty;
    bool ended = false;
    std::thread reader([&] { ended = !empty.pop(); });
    empty.wait_until_blocked();
    empty.close();
    reader.join();
    if (!ended) return 2;

    Channel full;
    for (int i = 0; i < 4; ++i) if (!full.push(i)) return 3;
    bool rejected = false;
    std::thread writer([&] { rejected = !full.push(4); });
    full.wait_until_blocked();
    full.close();
    writer.join();
    if (!rejected) return 4;
    for (int i = 0; i < 4; ++i) {
        auto value = full.pop();
        if (!value || *value != i) return 5;
    }
    if (full.pop()) return 6;
    std::cout << "fifo=50000; empty-close; full-close; joined" << std::endl;
}

```

运行与判据。上述统一命令中的 G7-D2 提取并处理本模块。先运行普通构建，再在 TSan 可用时运行插桩构建；需要指定输出和正常退出，超时为失败而非停机成功。

16.3 G7-D3 · 工具阳性对照：缺少同步的写冲突

命题、观察与边界。反例有意让两个线程无同步写入同一普通对象。只在隔离进程中启用 TSan 运行，要求 data race 诊断和指定退出码；不能把结果值、普通崩溃或超时算作成功。它验证工具对这个已知错误的检测能力，不验证 G7-D1/D2 的所有路径。

[反例 · 未定义行为：仅限 TSan 隔离检测 · G7-D3 · main.cpp]

```

#include <thread>
int value = 0;
int main() {
    std::thread writer([] { value = 1; });
    value = 2; // Deliberate data race: no HB edge between the two writes.
    writer.join();
}

```

运行与判据。上述统一命令中的 G7-D3 提取并处理本模块。先运行无竞争的 TSan 探针；可用时要求 data race 诊断与退出码 66。环境不可用明确 SKIP，不将普通崩溃充作检测。**协议论证边界。**D2 假设 mutex/CV 与标准容器正确、工作线程获得调度、内存分配不失败；它不覆盖任务执行异常、强制终

止、MPMC 无锁回收或 stop-token API。状态只在锁内转换，所有工作线程先 join 再销毁 channel。有限 stress/TSan 结果不能证明公平性或全程序无竞争；D3 阳性对照也不能提升这一结论。

17. Final Gate

下面这些问题应该能闭卷回答。

A. Data Race

1 为什么：

[机制片段 · 不承诺独立编译]

```
int x = 0;

// A
x = 1;

// B
use(x);
```

在没有 `synchronization` 时不能只说：“B 可能读到 0 或 1”？

2 为什么：sleep 1 second 不能建立 happens-before？

B. Happens-before

3 解释：

$SB + SW + transitivity \rightarrow HB$

4 为什么普通 payload 可以通过 atomic flag publication 安全跨线程？

C. Memory Order

5 什么场景适合：

`memory_order_relaxed`

？

6 为什么：

[机制片段 · 不承诺独立编译]

```
data = 42;
ready.store(true, relaxed);
```

不能配合 `relaxed load` 正确发布普通 `data`？

7 `acq_rel` 为什么主要自然出现在 RMW 上？

8 `seq_cst` 比 `release/acquire` 多提供的核心是什么？

D. Mutex / CV

9 为什么 `mutex` 保护的是 `invariant` 而不是 `variable`？

10 为什么：

[机制片段 · 不承诺独立编译]

```
if (!pred()) {
    cv.wait(lock);
}
```

通常错误？

11 为什么 notification 可以发生在 waiter 真正等待之前，而正确 predicate-based design 仍然不丢工作？

E. CAS

12 CAS failure 为什么修改 expected？

13 为什么：weak CAS 不等于：weak memory ordering？

14 为什么 atomic：

load + store

不能替代 CAS？

F. ABA / Reclamation

15 解释：

$A \rightarrow B \rightarrow A$

为什么 CAS 可能无法发现中间变化。

16 为什么：

Unlinked $\neq$ Safe to delete

？

17 Hazard Pointer 与 Epoch 的核心区别是什么？

18 为什么 generation/tagged pointer 不能自动解决 lifetime reclamation？

G. Queue

19 SPSC 为什么通常不需要 CAS 更新 head/tail？

20 SPSC 为什么需要两个方向的 release/acquire handoff？

21 MPSC 中为什么：reservation tail 不能直接等价于：published tail？

22 Per-slot sequence number 在 MPMC queue 中解决的核心是什么？

H. Thread Lifetime

23 为什么 joinable `std::thread` 析构会 terminate？

24 为什么 detach 不能解决 object lifetime？

25 为什么 `jthread.request_stop()` 不能强制杀死 thread？

26 为什么 thread member 通常应该最后声明？

I. Thread Pool

27 为什么 worker 不应该持 queue mutex 执行 Task？

28 为什么 bounded pool 中 worker 递归 blocking submit 可能 deadlock？

29 为什么 CPU-bound 与 blocking-I/O task 不应该机械使用同一个 pool？

30 为什么 work stealing 的主要架构价值不是“更复杂”，而是 common-case local ownership?

J. Architecture

31 为什么 single writer 能显著降低 concurrency complexity?

32 Sharding 的本质是什么? 不是: 多几个 queue。而是?

33 Immutable snapshot 主要优化什么 workload?

34 为什么: old snapshot no longer current 仍不代表可立即销毁?

35 为什么“每个字段都 atomic”经常不如一个 immutable snapshot?

18. Final Gate · 参考答案与常见误判

18.1 Data Race、HB 与内存序 (1 ~ 8)

1. 无同步的冲突非原子访问属于数据竞争，语言不把结果限定为旧值/新值二选一。还必须验证对象生命周期，而不只检查 load/store。
2. sleep 改变调度概率，不建立 synchronizes-with；必须由锁、原子发布或其他标准规定的同步机制建立关系。
3. 在线程内用 sequenced-before 排列相关操作，跨线程找出 synchronizes-with，再使用传递性得到所需 HB。箭头必须有规则依据，不能按墙钟时间补画。
4. 非原子 payload 的写与读可由单次 release/acquire 发布建立 HB。读者必须读到相应发布；写者不能在读者使用时再次改写同一 payload。
5. 独立计数且不依赖该计数发布其他数据时可使用 relaxed；还要分析计数溢出、复合不变量和对象存活。
6. relaxed 的读写没有所需发布同步，标志观察本身不使普通 data 读写合法。
7. RMW 既读取此前状态又发布本次更新，可能同时需要 acquire 和 release；纯 load/store 不需要也不接受所有 RMW 组合。
8. seq_cst 提供对相应 SC 操作的受约束总序，不会把其他错误协议变正确；混合弱序操作须另行推理。

18.2 Mutex、CV 与 CAS (9 ~ 14)

9. 相关字段组成一个状态不变量；只给各字段分别上锁可能仍让读者观察到不允许的组合。
10. wait 可伪唤醒，醒来后谓词也可能已被其他线程改变；使用循环或谓词重载，并在同一锁协议下检查状态。
11. 不是因为通知被保存，而是谓词状态受锁保护并持久存在。消费者持锁检查和 wait 的原子解锁/等待配合生产者的持锁修改，消除丢失工作的窗口；锁外 atomic 标志加 notify 不能机械替代。
12. 失败意味着当前值不同或 weak 伪失败；expected 接收观察结果，下一次 desired 应由它重算。不能反复用旧假设提交状态。
13. weak 指允许伪失败，内存序指定同步/排序，二者独立。
14. 独立原子 load 和 store 之间可插入另一修改；CAS 把条件检查与更新合成一个原子转换。

18.3 ABA、回收与队列 (15 ~ 22)

15. 比较当前值表示不能看到 A→B→A 的历史，因此相同表示不保证相同代次。
16. 从结构摘除只阻止某些新获取，不能证明此前读者已不再使用对象。
17. Hazard pointer 公开保护具体对象并要求正确的获取/重验协议；epoch 等待相关读者离开旧时期后批量回收。读者停顿、注册成本与回收延迟的权衡不同。
18. tag 可区分某些历史变化，却不阻止对象被释放；还要处理回绕并提供真正的存活保护。
19. 常见 SPSC 中每个游标各有唯一写者，不需要多个写者竞争同一个增量；交叉观察仍要同步。
20. 正向发布元素供消费，反向发布消费完成供生产者安全复用槽位；缺少反向边会让下一轮写覆盖仍在读取的数据。

21. 取得位置不等于元素已经构造完成；多生产者可按不同速度完成，消费者不能跨过未发布位置。
22. 常见有界环中每槽序号区分空闲、就绪与复用代次；这只是协议组成，不是所有 MPMC 算法或内存回收的通用证明。

18.4 线程生命周期与运行时 (23 ~ 30)

23. joinable thread 的析构按标准调用 `terminate`；不能偷偷把仍运行的工作和依赖责任丢掉。
24. `detach` 只分离线程句柄的等待责任，不延长捕获对象生命周期，也不建立停机确认。
25. `stop_token` 是合作机制；工作函数与阻塞操作必须响应请求。不能依靠强杀跳过不变量恢复。
26. 最后声明通常先析构，可使 `jthread` 先于依赖销毁；仍须满足唤醒、退出、`join` 和锁依赖，声明顺序不是完整方案。
27. 在队列锁内执行任务会串行化调度，并可能被回调重入、阻塞和再次提交拖入死锁。
28. 所有 `worker` 都等待有界队列空位时，可能已没有线程能消费腾出位置。需非阻塞/拒绝、`helping` 或不同依赖拓扑。
29. 阻塞 I/O 占住执行槽，会饿死需要 CPU 或完成回调的工作；线程数、队列和资源隔离须按负载决定。
30. `Work stealing` 保留常见路径的本地队列操作，仅在需要时跨线程协调；仍有窃取、任务依赖与停机复杂度，并非自动更快。

18.5 架构 (31 ~ 35)

31. 单写者集中状态转换和不变量；其他线程的读取仍需同步或快照，不能据此无锁读取可变状态。
32. 分片按稳定的 `ownership key` 划分写入权限、执行位置与关联数据；跨片事务、迁移和热点仍须协议，不只是多建几个队列。
33. 不可变快照适合读多写少、允许一定陈旧度的负载，以发布和版本保留换取简单读取。
34. “非当前”不等于无人持有；必须等全部相关读者释放或到达回收安全点。
35. 各字段 `atomic` 不自动构成一致版本。快照把相关字段作为一个逻辑版本发布，但需要真正不可变及正确生命周期管理。

19. G0 ~ G7 的统一模型

下面只列跨章审查的交接点，概念定义回到各章主讲处；本节保留它们在并发问题中的用途，不另写一套定义。

19.1 G1 — Object Model

由 G1 §1 ~ 2 检查存储、生命周期、类型、边界与对齐，先确认访问合法，再讨论同步。

19.2 G2 — Ownership

由 G2 §1、§9 列出拥有者、借用者、交接与清理；并发新增的问题是使用者是否真的都已退出。

19.3 G3 — Value Semantics

由 G3 §4、§8 区分复制、移动、分配与表示，不由指针大小推断跨线程交接的完整成本。

19.4 G5 — Genericity

由 G5 §9 界定哪些变化留在编译期，避免调度与生命周期策略无必要地扩散成模板参数。

19.5 G6 — Performance

由 G6 §8 ~ 9 分析布局、缓存、分配、TLB、分支和一致性成本；这些观察不替代本章的语言顺序论证。

19.6 G7 — Concurrency

- Who may access or mutate this state concurrently,
- and what ordering/lifetime guarantees make that legal?

最终统一：

```
Object
  ↓
Lifetime
  ↓
Ownership
  ↓
Representation
  ↓
Machine Cost
  ↓
Mutation Authority
  ↓
Synchronization
  ↓
Concurrent Architecture
```

这已经是一套相当完整的 systems reasoning model。

20. 实际代码的跨层审查

看到：

[机制片段 · 不承诺独立编译]

```
std::shared_ptr<MutableModel> model;
std::mutex mutex;

void worker() {
    std::lock_guard lock{mutex};
    model->update(...);
}
```

以前可能只问：“mutex 会不会慢？”现在应该按完整链路分析：

G1

`model pointee lifetime` 是否有效？

G2

为什么是 `shared ownership`？

G3

`shared_ptr copy/refcount` 成本是什么？

G6

`model working set` 是否在 `cores` 间迁移？

`mutex/cache-line` 是否 `contention`？

G7

为什么有多个 `writers`？

能否 `single writer`？

能否 `shard by model/entity`？

`readers` 是否可以使用 `immutable snapshot`？

这项分析的目的在于：把 **synchronization problem** 向上提升成 **ownership architecture problem**。

21. 本章范围与后续阅读

数据竞争、内存序、锁与等待、CAS、回收、队列、线程停机、任务运行时和并发架构均保留为本章主题。协议论证是明确假设下的工程推理，不是全程序形式化验证。下一阶段按总导航进入 **G8 ABI** 与 **C** 互操作；接口一旦交给独立组件，线程与销毁合同也必须穿过边界。需要完整关闭案例时回查 **G10 §7**。

22. 参考与验证入口

全系列导航 · 实验说明 · 本批修订与证据

语言模型参见 **N4950** 数据竞争、原子内存序；工具边界参见 **Clang ThreadSanitizer**。